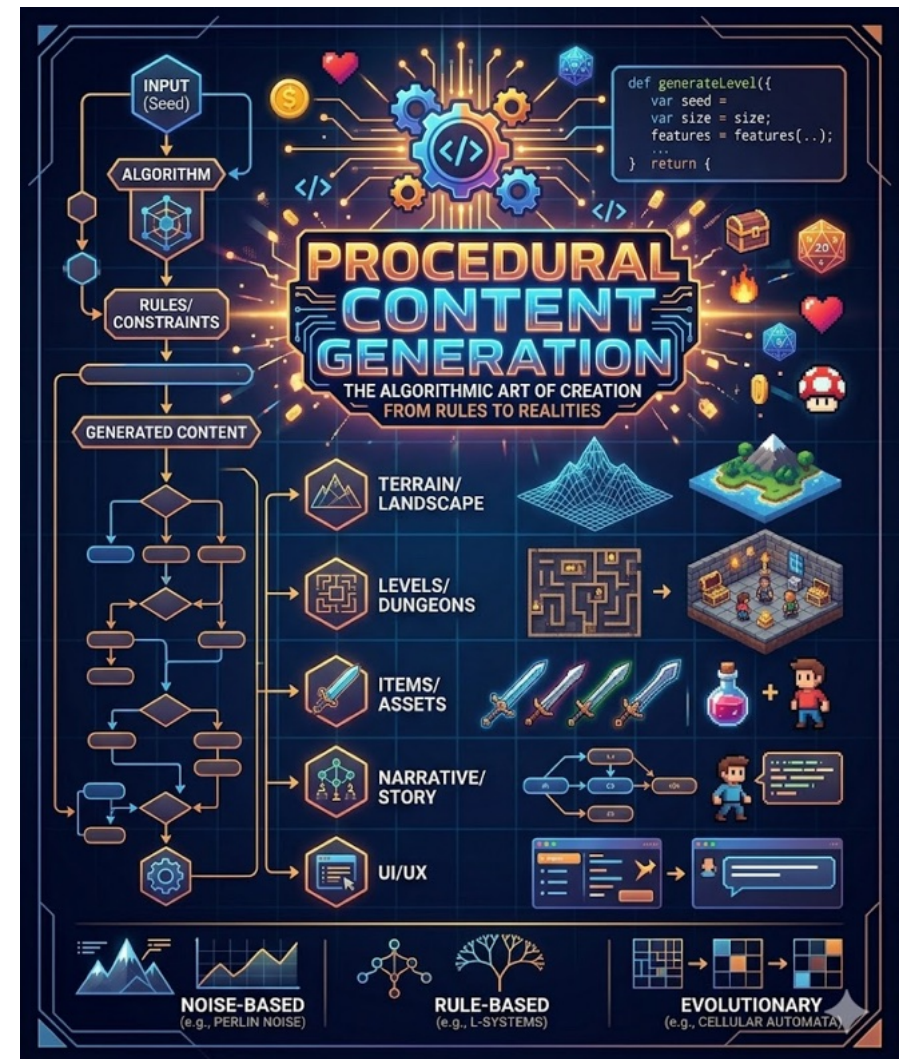


Lecture 4

Procedural Content Generation



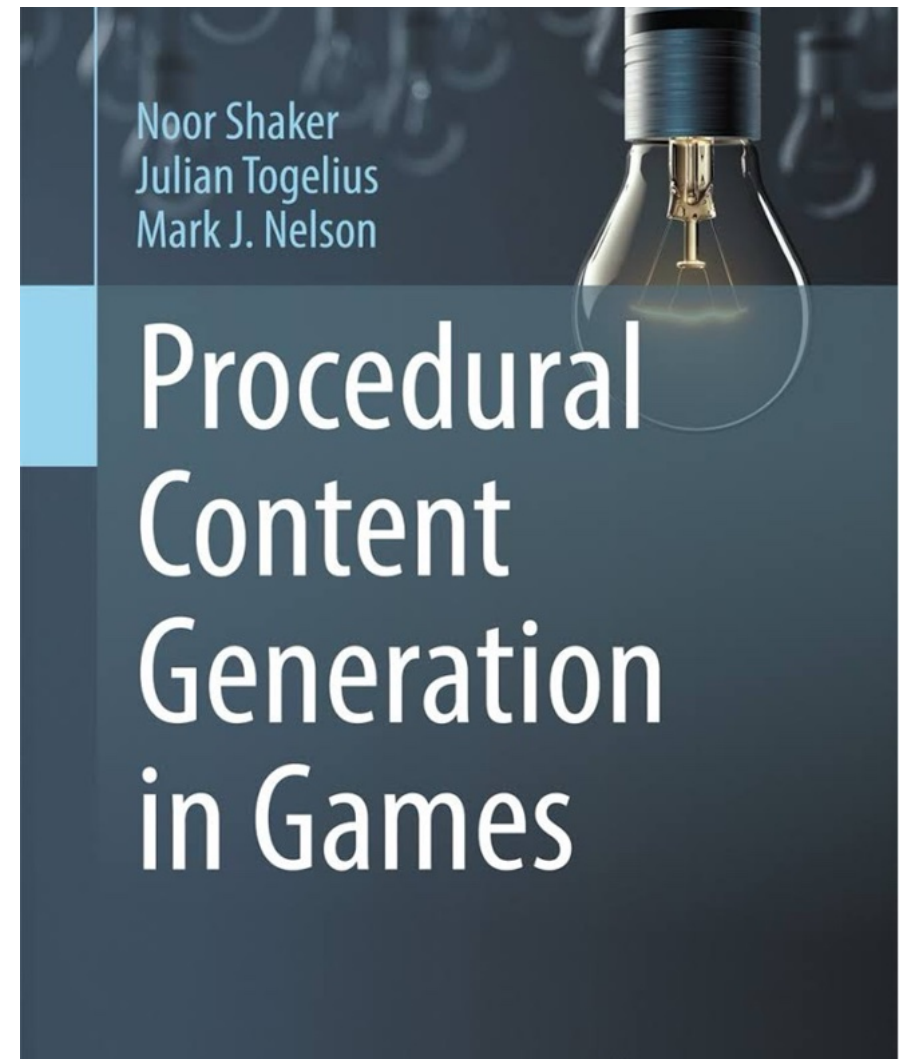
References

Procedural Content Generation in Games

Noor Shaker, Julian Togelius,
Mark J. Nelson, 2016

ISBN-13: 978-3319427140

Jonas Freiknecht, Procedural Content Generation for Games, PhD Thesis, Universität Mannheim, 2020

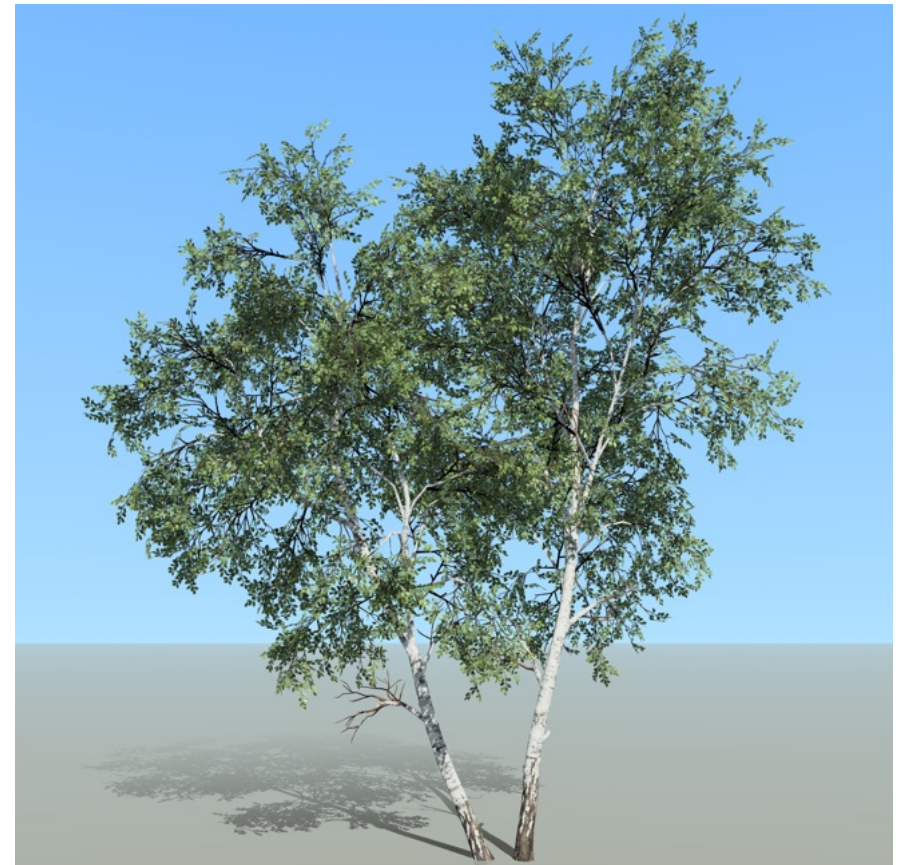


What is Procedural Content Generation (PCG)?

- “...PCG refers to computer software that can create game content on its own, or together with one or many human players or designers”

Shaker et al.

- Short 🖥️: What's procedural generation?



[SpeedTree, 2020]

What is Procedural Content Generation (PCG)?

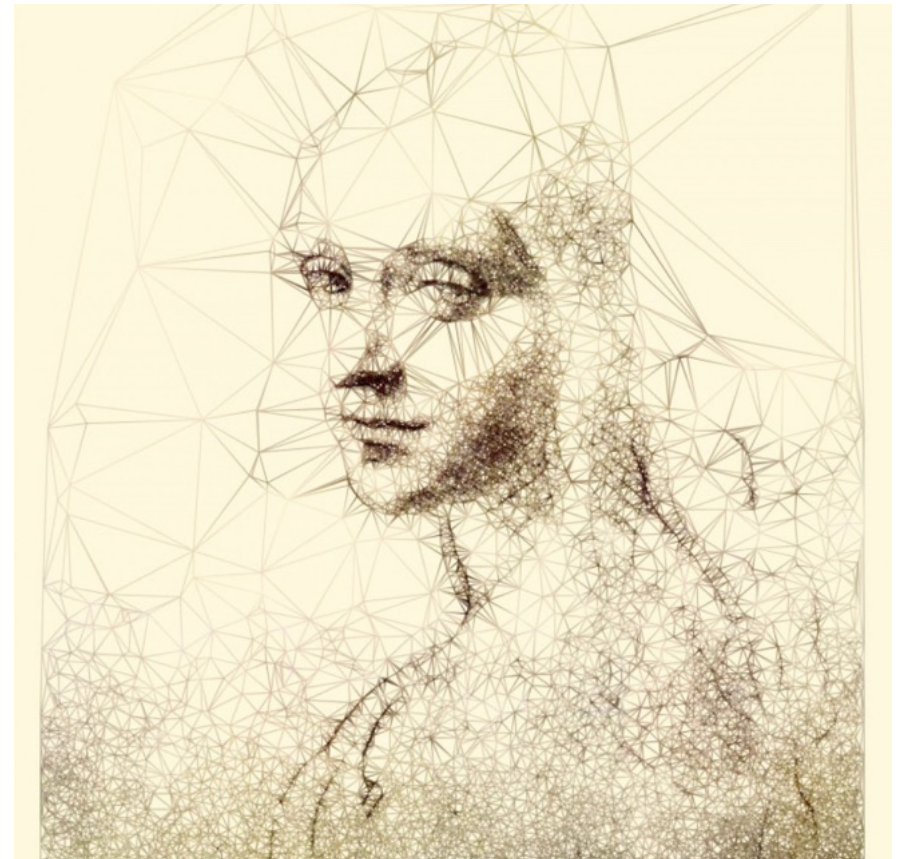
- PCG uses algorithms to create game content automatically
- Contrasts with manual creation by artists and designers
- Focus on scalability: generating large amounts of content efficiently
- Enhances replayability: fresh experiences each playthrough



[Townscaper, 2020]

PCG vs. Manual Art Creation

- Manual Art: crafted by human artists, unique and curated
- PCG: algorithm-driven, can produce vast, diverse content
- Manual excels at detail, PCG excels at volume and variation
- Hybrid approaches often combine both for best results



[[Delaunified Portraits](#)]

Tradeoffs of PCG

- Benefits:
 - **Scalability:** easily produce large game worlds, assets, and scenarios
 - **Replayability:** each playthrough can differ, keeping the game fresh
 - **Resource Efficiency:** reduces need for exhaustive manual content creation
 - **Dynamic Content:** adapt game environments based on player actions
- Drawbacks:
 - **Processing Time:** Real-time generation may require significant CPU/GPU processing power during loading or gameplay
 - **Repetitiveness:** The content may seem repetitive, generic or lacking the narrative intent of a human designer
 - **Limited Control:** It is difficult to ensure that the outcome is what was intended



[SceneCity, Blender addon]

More PCG Pointers

- GDC Talk: [Practical Procedural Generation for Everyone](#)
- Playlist: [A Beginners Guide To Procedural Generation](#)
- GDC [Best Practices for Procedural Narrative Generation](#)

No Man's Sky: Case Study

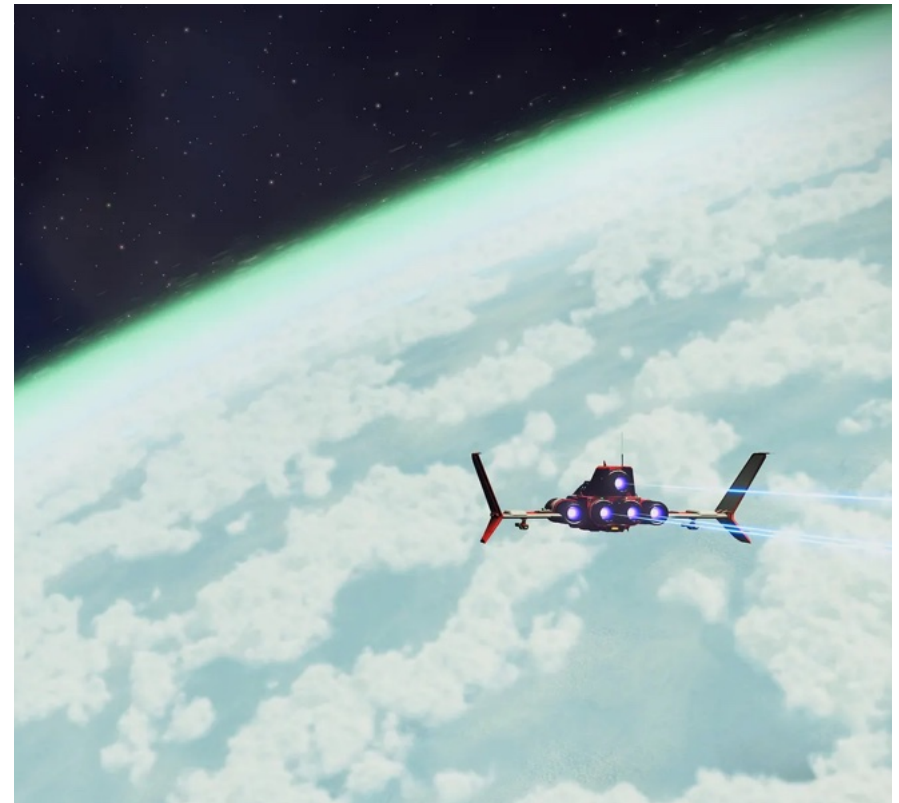
- Exploring infinite worlds using a 64-bit seed.
- Finite code enabling endless procedural generation.
- A perfect example of procedural content generation (PCG) in games.



[Image Source]

The Power of the 64-bit Seed

- What is a 64-bit seed?
 - A deterministic starting number for PCG algorithms.
 - Ensures consistent, repeatable world generation.
- Infinite Exploration:
 - Generates billions of unique planets.
 - Unique flora, fauna, landscapes, and weather.
- Finite Code:
 - Algorithms define rules and parameters.
 - Seed acts as a key to unlock enormous diversity.



[Image Source]

Implications for Game Design

- Benefits:
 - Endless replay value
 - Less manual asset creation
 - Dynamic, adaptive game worlds
- Challenges:
 - Maintaining narrative coherence
 - Avoiding repetitive content
 - Balancing algorithms with artistic vision

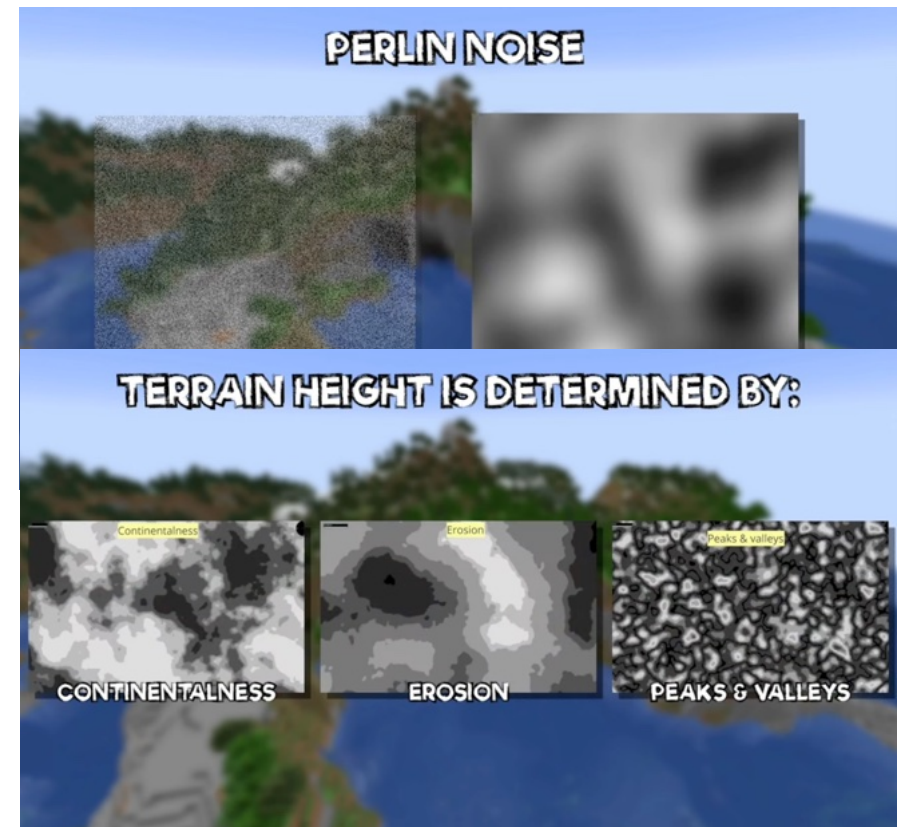
Minecraft: Case Study

- Chunk-based generation and heightmap-driven terrain
- Three-Dimensional grid of cubes
- Terrain Generation, Biomes and Climate
- Determinism
- Structure Spawning
- Video: [How Does Minecraft Generate Worlds?](#)



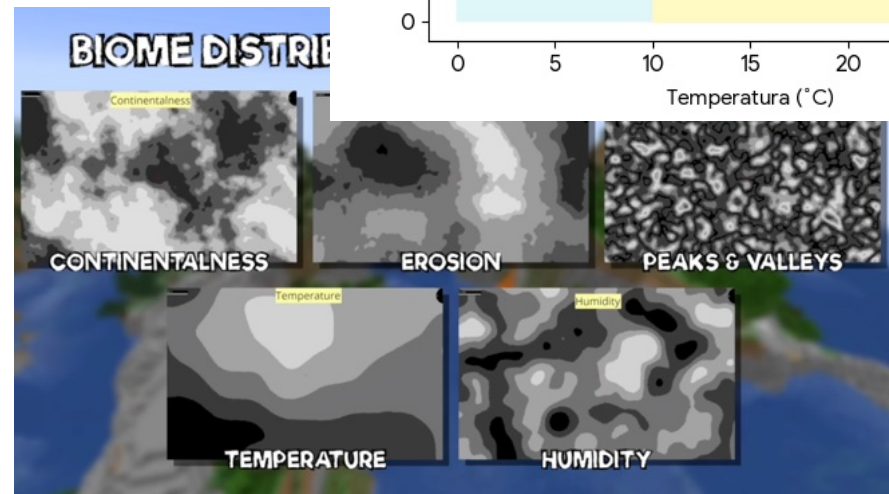
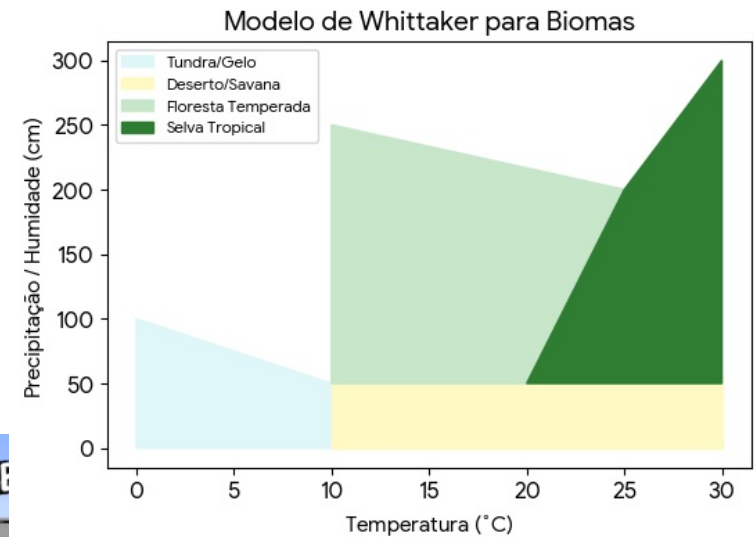
Terrain Generation

- 3D Perlin Noise algorithm
- Creates smooth mountains and valleys
- Continental-ness, Erosion, Peaks & Valleys maps



Biomes and Climate

- Noise maps: temperature, humidity, height
- Whittaker diagram for biomes
- Generates deserts, forests, oceans



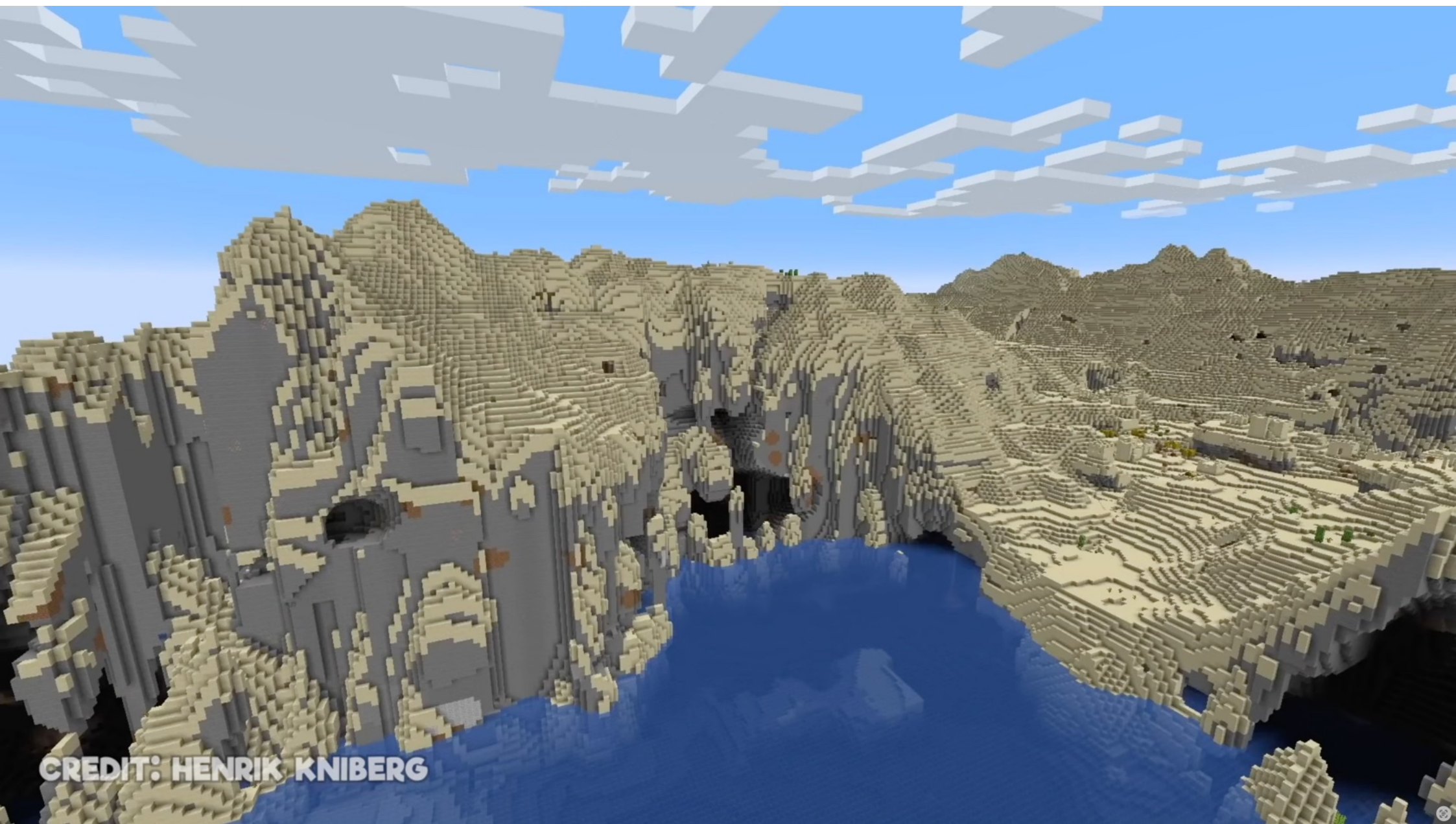
Deterministic Seeds

- Seed determines world layout
- Reproducible worlds with same seed
- Easy sharing of landscapes

Structure Spawning

- Procedural trees and ores
- Populates villages, dungeons
- On-demand chunk spawning





CREDIT: HENRIK KNIBERG



CREDIT: HENRIK KNIBERG



CREDIT: HENRIK KNIBERG

Types of PCG

- **Terrain Generation:** Creating landscapes or environments algorithmically, often for video games or simulations.
- **Level Generation:** Generating game levels dynamically, including layouts, objects, enemies, and obstacles.
- **World Building:** Procedurally generating entire virtual worlds or universes, including geography, climate, flora, and fauna.
- **Texture Generation:** Creating textures algorithmically for use in graphics, such as terrain or surface textures.
- **Procedural Narrative Generation:** Generating storylines, quests, dialogue, or other narrative elements dynamically.

Types of PCG

- **Character Generation:** Creating characters with unique appearances, behaviors, abilities, and personalities procedurally.
- **Music and Sound Generation:** Generating music or sound effects algorithmically, enhancing interactive experiences.
- **Item Generation:** Creating items, weapons, or equipment dynamically, with varying properties and rarities.
- **Puzzle Generation:** Procedurally generating puzzles or challenges for games or interactive media.
- **Procedural Animation:** Creating animations algorithmically, adding dynamism and realism to interactive content.

Techniques Used in PCG

- **Random Number Generator:** Fundamental for creating randomness in content generation.
- **Noise Functions:** Smoothly varying pseudo-random values used for terrain generation and texture synthesis.
- **Cellular Automata:** Grid-based models simulating natural phenomena like terrain formation and vegetation growth.
- **L-Systems:** Formal grammars describing growth processes, commonly used for generating organic shapes and vegetation.
- **Dungeon Generation:** Various algorithms (e.g., cellular automata, random walk, BSP) for creating procedural dungeon layouts.

Techniques Used in PCG

- **Markov Chains:** Statistical models generating content sequences (e.g., text, levels) with predefined properties.
- **Generative Adversarial Networks (GANs):** Machine learning models for generating realistic content like images or textures.
- **Iterative Refinement:** Techniques (e.g., simulated annealing, genetic algorithms) for gradually improving generated content.
- **Scripted Generation:** Content generation based on predefined scripts or rulesets, offering precise control over output.
- **Procedural Grammar Systems:** Formal grammar rules for generating complex structures, narratives, or content.
- **LLMs:** Using large language models in combination with any grammar-based algorithm.

Togelius et al. Taxonomy for PCG

- **Online vs offline** → During the game live or during level design phase
- **Necessary vs optional**
- **Degree and dimensions** of control → How many parameters
- **Generic vs adaptive** [to the player]
- **Stochastic vs deterministic** → random or can rebuild equal
- **Constructive vs generate-and-test** → 1 go or tries several times
- **Automatic** generation vs mixed **authorship**

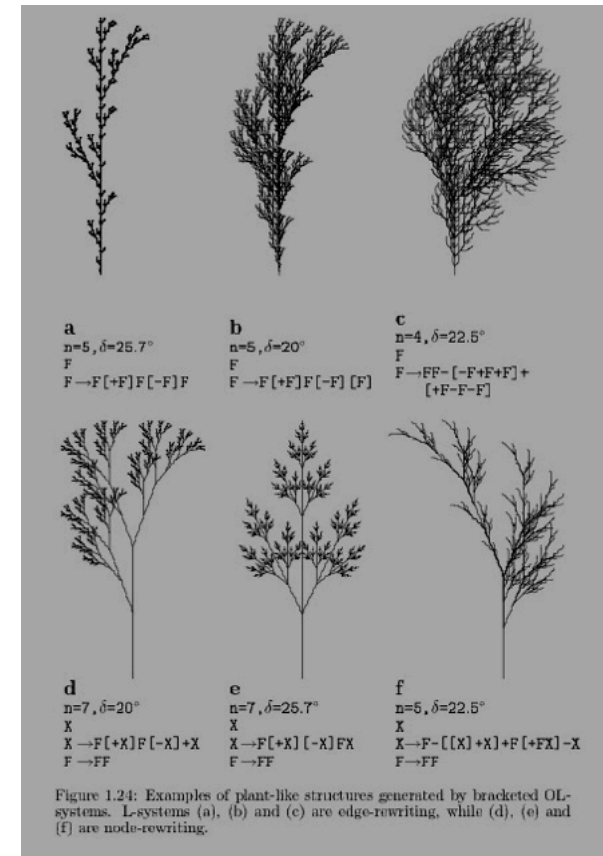
More on some PCG Techniques

- Topics we will discuss in more detail:
 - L-Systems → Vegetation
 - Cellular Automata → World Building
 - Dungeon Generation Algorithms → Level Building
 - Noise → Textures and Terrain Building

Recursive Structures

L-Systems

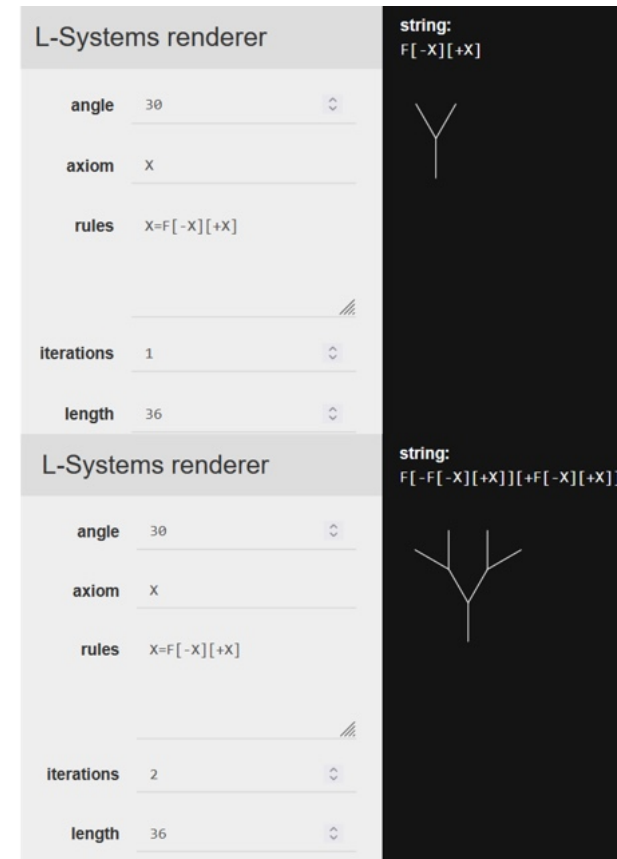
- Developed by Aristid Lyndenmayer in 1968
- Formal grammar systems used to model the growth process of plants and other branching structures
- Uses **Axioms, Production Rules** and an **Alphabet**.
 - **Variables: Symbols** in the alphabet that are substituted (have a production rule)
 - **Constants/Terminals: Symbols** in the alphabet that don't get substituted (no rules)
- Applications: Vegetation, geometric shapes and fractals.



[Image source]

L-Systems

- **Axiom:** initial state of the System
 - Ex: X
- **Production Rules** are recursive rewriting expressions:
 - Ex: $X = F[-F[-X][+X]][+F[-X][+X]]$
- **Alphabet:**
 - Lines: **X**, **F**
 - Rotations: **+**, **-**
 - Branches: **[]** (equivalent to push **[** and pop **]**), after **]** returns to where it was before **[**



[PCG using L-Systems]

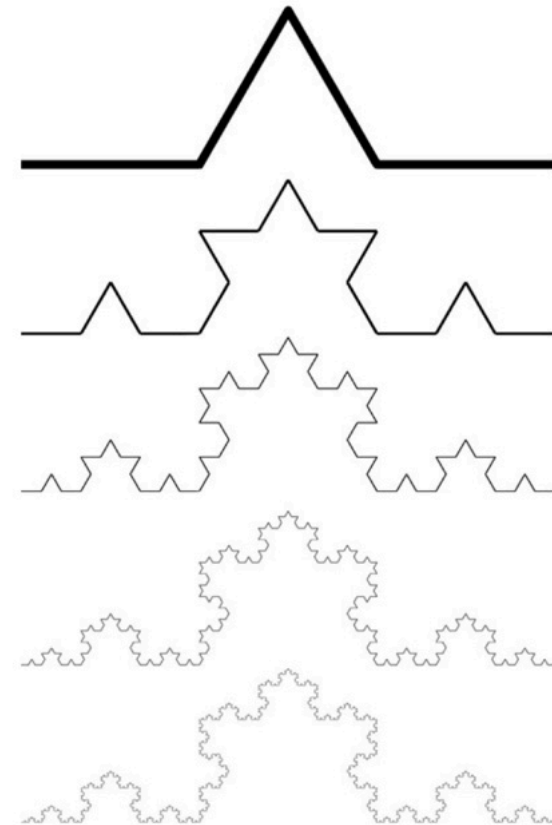
L-Systems

- Extension to 3D: Symbols interpreted similarly to Turtle Graphics commands by moving a “cursor”: Forward, Pitch, Yaw, Roll.
- Stochastic L-Systems: Add probability (P) to rules to create organic variation.
- Applications: Trees, Plants, Roads, Structures, ...

Feature	Deterministic (Symmetrical)	Stochastic (Natural)
Rules	1 Rule per symbol ($F=Result$)	Multiple rules with weights (P)
Output	Identical every time	Unique each time
Visual Style	Perfectly symmetrical / Recursive	Irregular / Organic
Game Use	Architected patterns, Crustals	Trees, plants, lightning, veins

Examples: Koch Curve

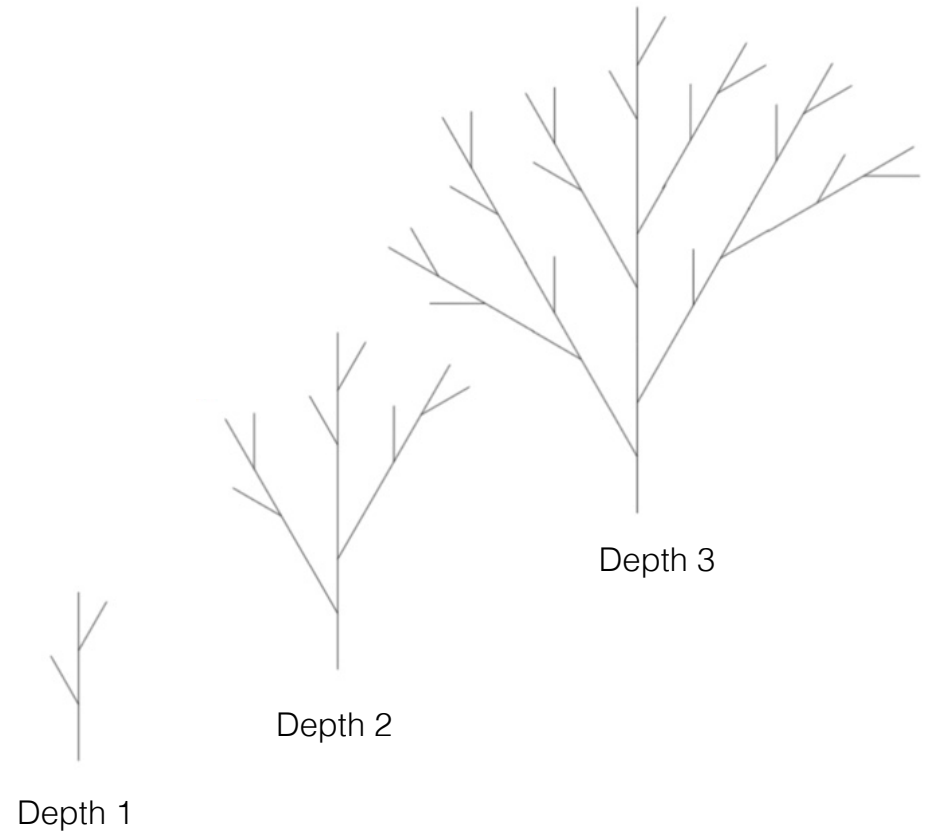
- Rule: $F = F - F ++ F - F$
- **F**: Draw line segment scaled by $1/3$
- **-**: Turn left 60°
- **+**: Turn right 60°



[Credits: [UTexas](#)]

Examples: 2D Tree

- **F** (Move forward)
- **L₀, L₁, L₂** (Draw branches)
- **T** (Draw terminating leaf)
- **+** (Turn Right)
- **-** (Turn Left)
- Axiom: **L₀**
- Rules:
 - $L_0 \rightarrow F [-FL_1] F [+FL_2] FL_0$ (center branch)
 - $L_1 \rightarrow F [-FL_1] F [+FT] FL_1$ (left half branch)
 - $L_2 \rightarrow F [-FT] F [+FL_2] FL_2$ (right half branch)



Parameterized L-Systems

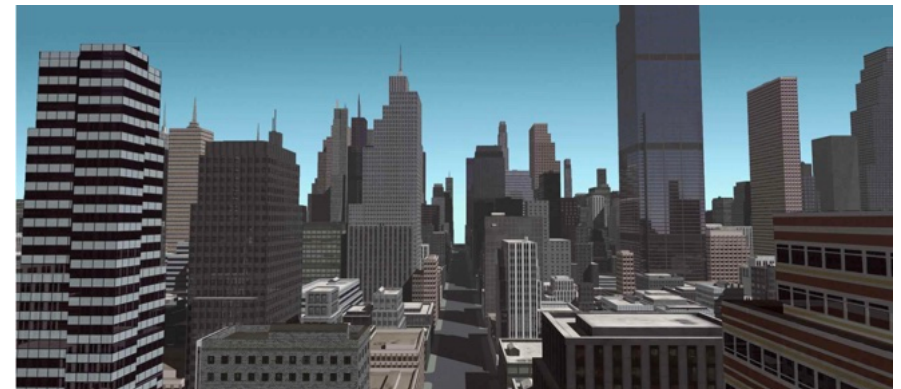
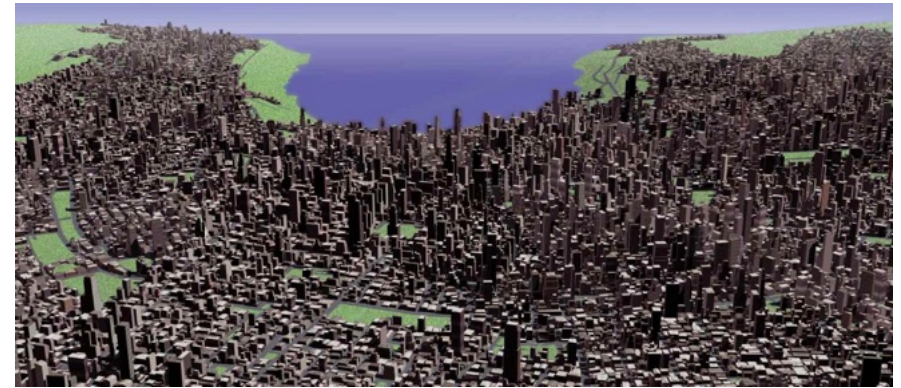
- Action specified by symbol can be parameterized
 - Ex: Rotate α degrees right, Draw segment reduced by s
- Randomized rule-selection
- Parameterization based on depth
- Changes in parameters over time



[From [Wikipedia](#)]

Example: Generating Cities

- Application of L-Systems to City Generation
- See Procedural Modeling of Cities paper for details



Iterated Function Systems (IFS)

- An IFS is:
 - Collection of affine transformations (scaling, rotation, translation) applied to an original shape.
 - Each transformation has a probability associated
 - Transformations must be contractive
 - The fractal is the “fixed point” or “attractor” of the combined mappings.

$$f_w(x, y) = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} e \\ f \end{bmatrix}$$

General form of a mapping*

* easily extended to 3D

$$f_1(x, y) = \begin{bmatrix} 0.00 & 0.00 \\ 0.00 & 0.16 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$f_2(x, y) = \begin{bmatrix} 0.85 & 0.04 \\ -0.04 & 0.85 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0.00 \\ 1.60 \end{bmatrix}$$

$$f_3(x, y) = \begin{bmatrix} 0.20 & -0.26 \\ 0.23 & 0.22 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0.00 \\ 1.60 \end{bmatrix}$$

$$f_4(x, y) = \begin{bmatrix} -0.15 & 0.28 \\ 0.26 & 0.24 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0.00 \\ 0.44 \end{bmatrix}$$

Barnsley Fern mappings

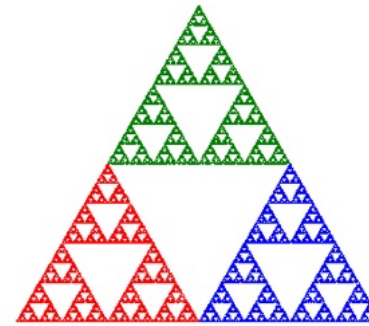
L-Systems vs IFS

- L-Systems and Iterated Function Systems approach fractal generation from different angles:
 - L-Systems use “language”
 - IFS use “math”

Feature	L-Systems	Iterated Function Systems (IFS)
Primary Tool	Formal Grammars (Strings)	Linear Algebra (Matrices)
Growth Style	Step-by-step expansion	Global contraction
Best For	Plants, trees, branching structures	Clouds, mountains, classic fractals
State Management	Uses a "Stack" (memory)	Uses "Mappings" (transformations)
Visual Logic	"Move here, then draw this"	"The whole is made of these parts"

IFS examples

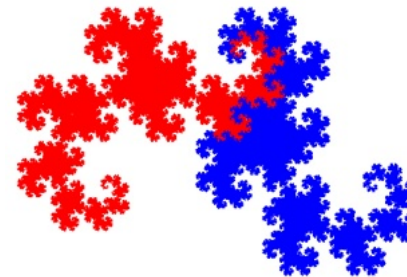
Shape	Map (w)	a	b	c	d	e	f	Probability (p)
Sierpinski Triangle	1	0.5		0	0.5		0	0.33
<i>(Equilateral variant)</i>	2	0.5		0	0.5	0.5		0.33
	3	0.5		0	0.5	0.25	0.5	0.34
---	---	---	---	---	---	---	---	---
Barnsley Fern	1		0	0	0.16		0	0.01
<i>(Biological model)</i>	2	0.85	0.04	-0.04	0.85		0.16	0.85
	3	0.2	-0.26	0.23	0.22		0.16	0.07
	4	-0.15	0.28	0.26	0.24		0.044	0.07
---	---	---	---	---	---	---	---	---
Heighway Dragon	1	0.5	-0.5	0.5	0.5		0	0.50
<i>(90° turns)</i>	2	-0.5	-0.5	0.5	-0.5	1		0.50



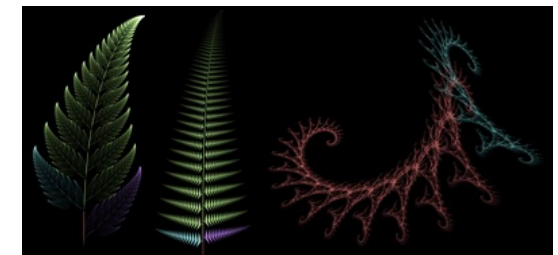
[Sierpinsky Triangle]



[Barnsley Fern]

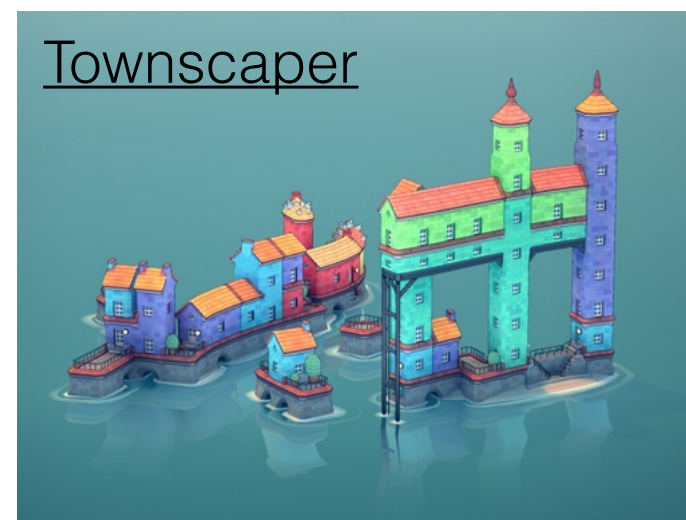


[Heighway Dragon]



[WebGL IFS Demo]

Wave Function Collapse



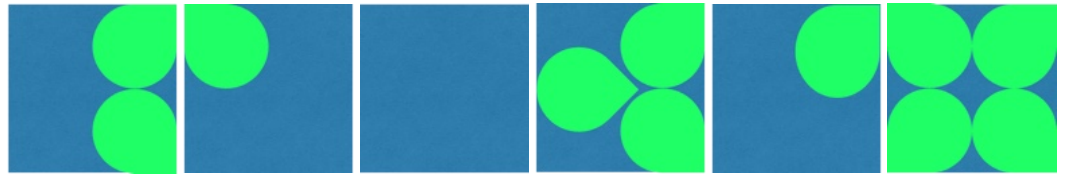
Wave Function Collapse

- A constraint programming technique to choose between tile variations

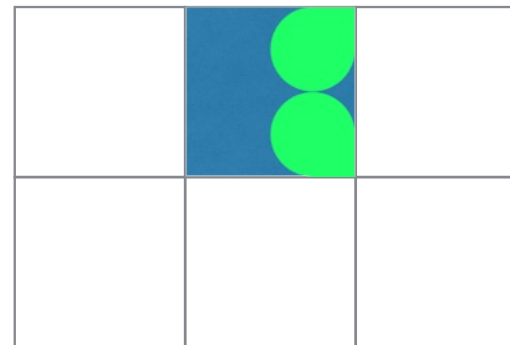
- Steps:

1. Define grid and initial cell
2. Define constraints
3. Collapse the initial cell

[Library]



[Grid]



[Constraints]

Keep color continuity
No rotations

Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

3. Collapse the initial cell

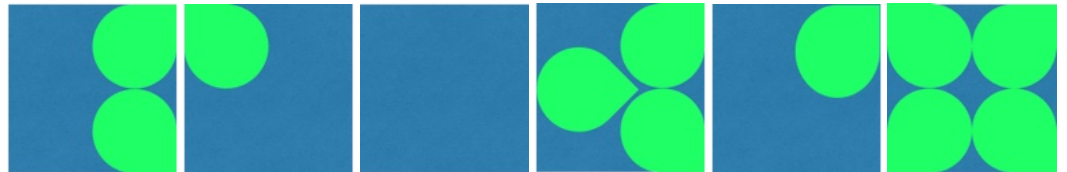


4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)

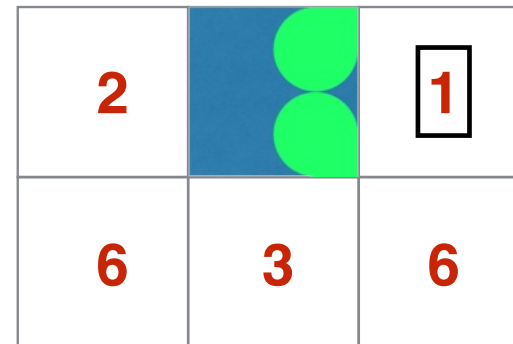
5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled

[Library]



[Grid]



[Constraints]

Keep color continuity
No rotations

Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

3. Collapse the initial cell

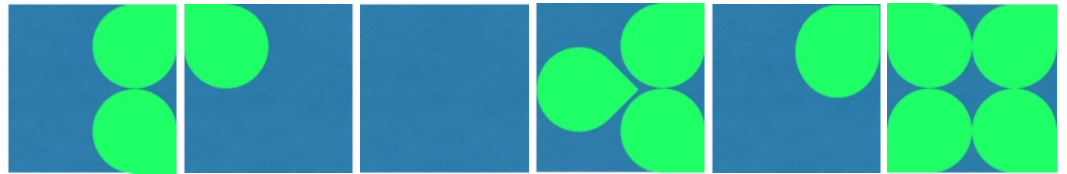
4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)



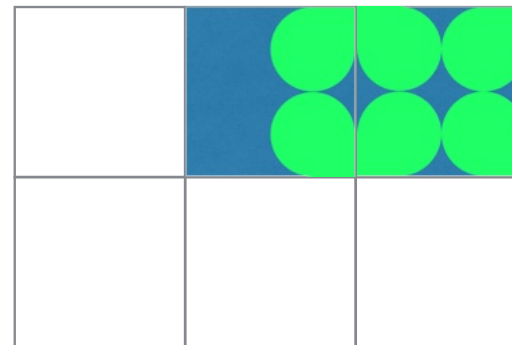
5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled

[Library]



[Grid]



[Constraints]

Keep color continuity
No rotations

Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

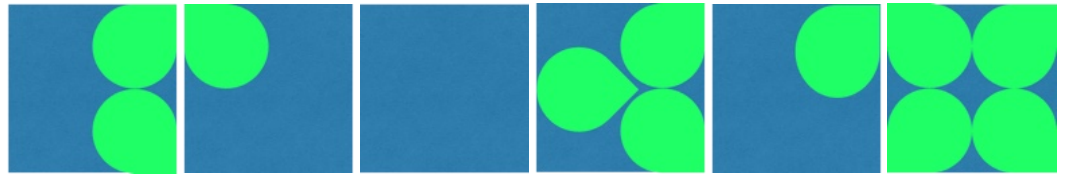
3. Collapse the initial cell

4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)

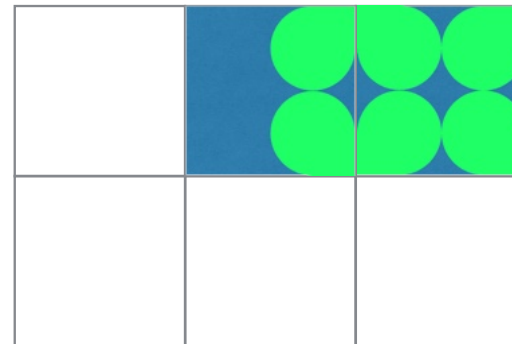
5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled

[Library]



[Grid]



[Constraints]

Keep color continuity

No rotations



Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

3. Collapse the initial cell

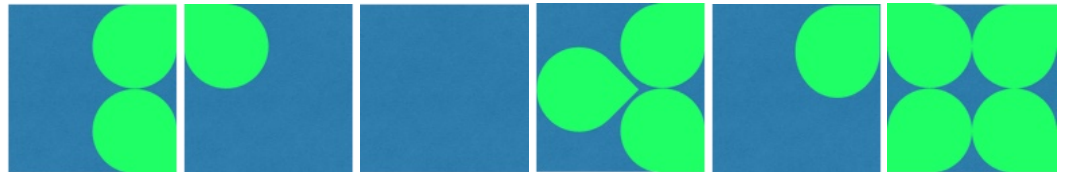


4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)

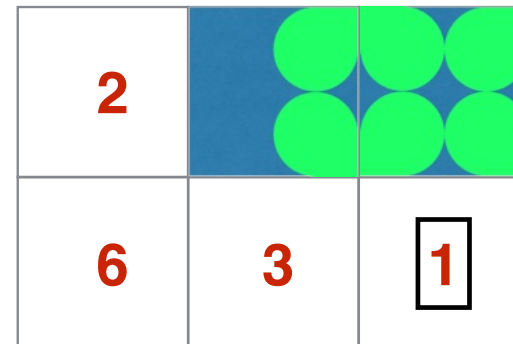
5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled

[Library]



[Grid]



[Constraints]

Keep color continuity
No rotations

Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

3. Collapse the initial cell

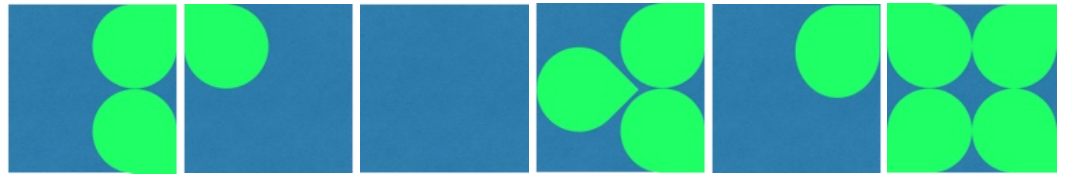
4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)



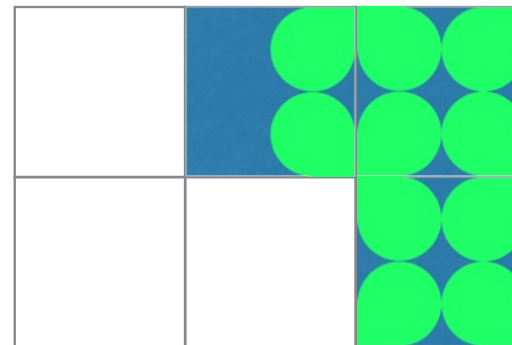
5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled

[Library]



[Grid]



[Constraints]

Keep color continuity

No rotations

Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

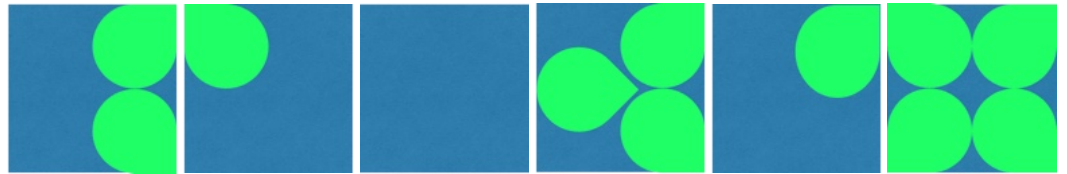
3. Collapse the initial cell

4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)

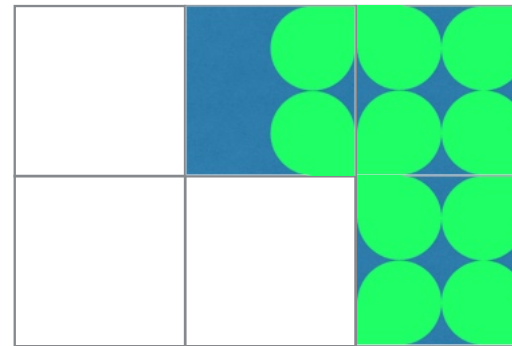
5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled

[Library]



[Grid]



[Constraints]

Keep color continuity

No rotations



Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

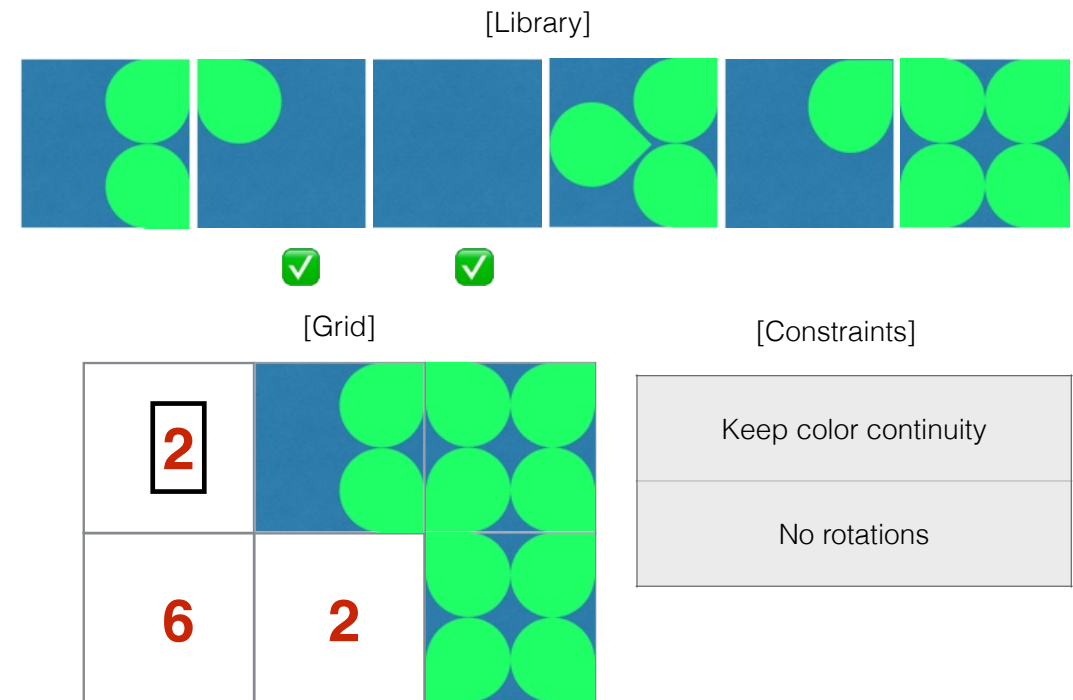
3. Collapse the initial cell



4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)

5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled



Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

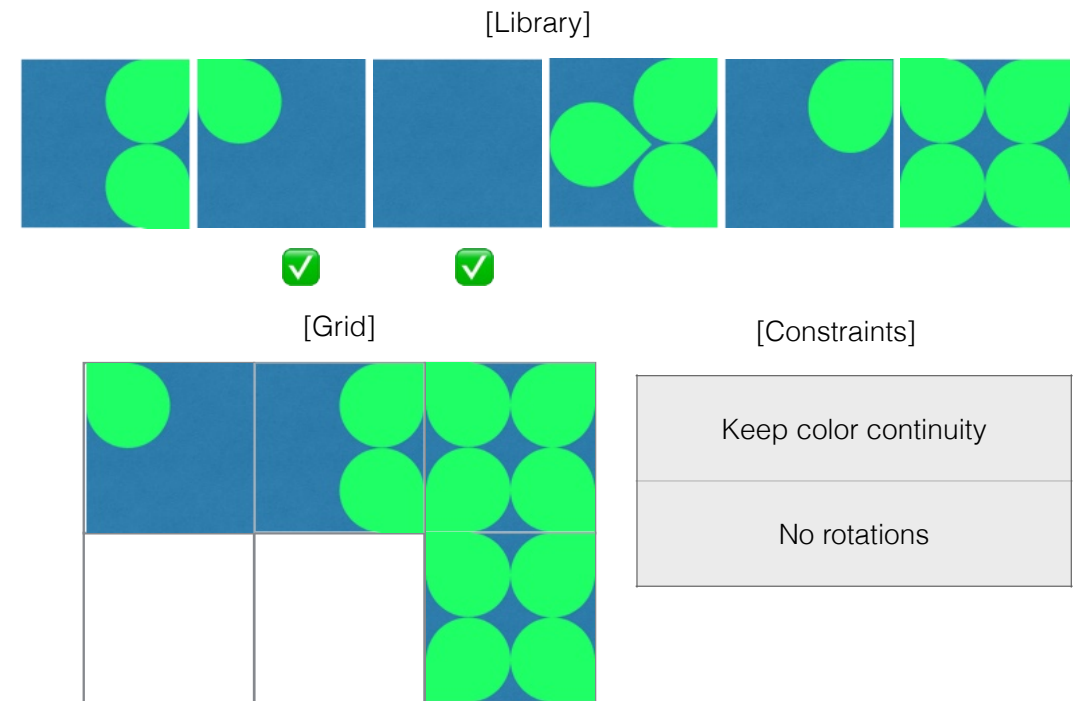
3. Collapse the initial cell

4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)



5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled



Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

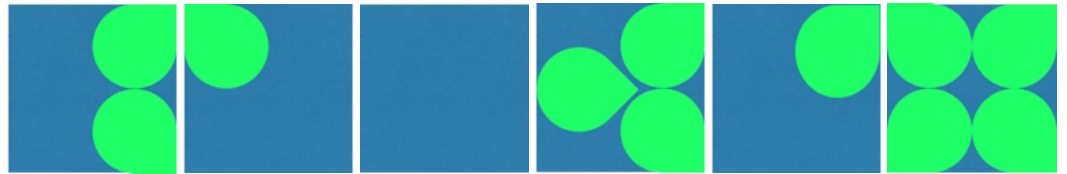
3. Collapse the initial cell

4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)

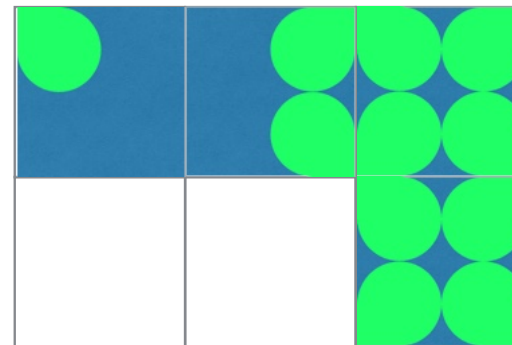
5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled

[Library]



[Grid]



[Constraints]

Keep color continuity

No rotations



Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

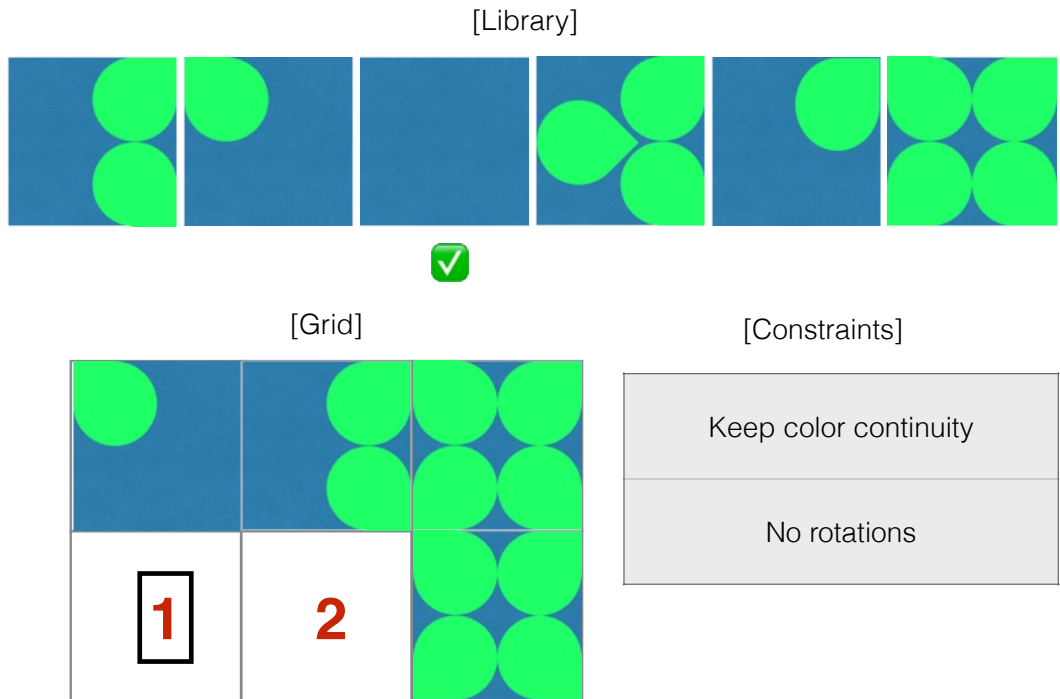
3. Collapse the initial cell



4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)

5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled



Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

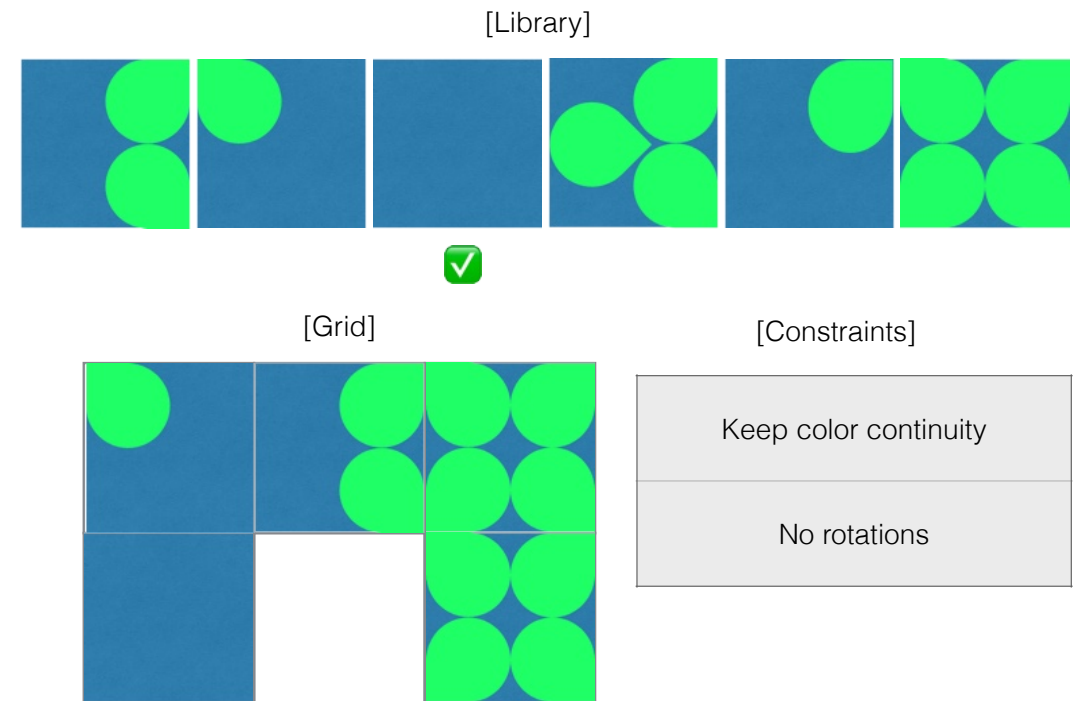
3. Collapse the initial cell

4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)



5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled



Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

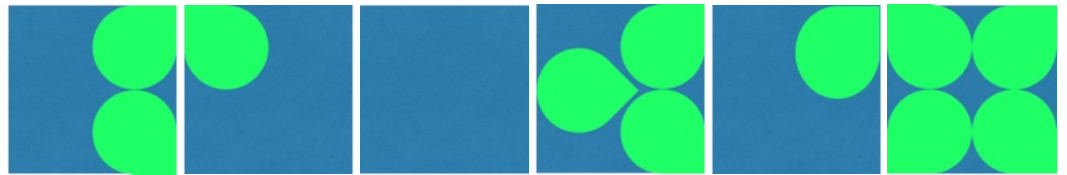
3. Collapse the initial cell

4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)

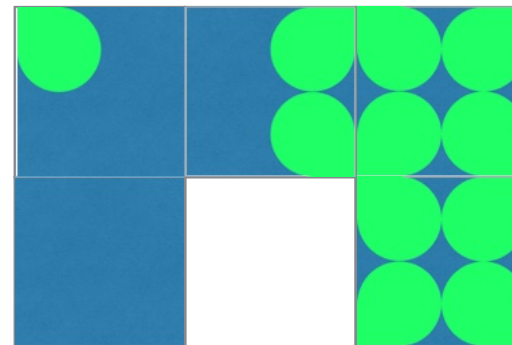
5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled

[Library]



[Grid]



[Constraints]

Keep color continuity

No rotations



Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

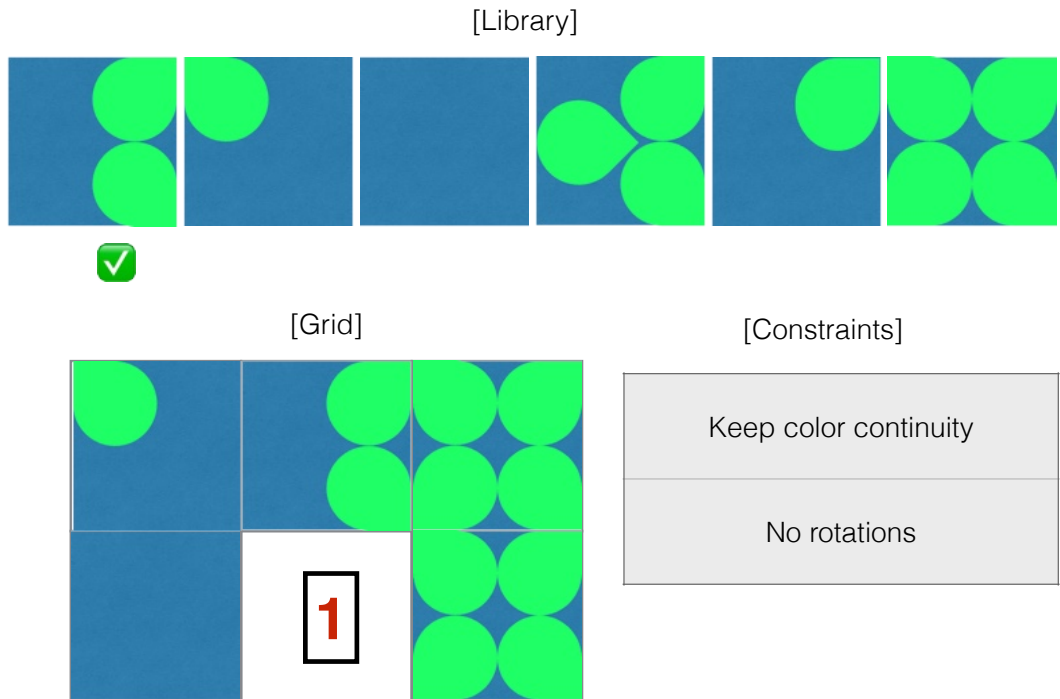
3. Collapse the initial cell



4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)

5. Collapse cell by assigning one of the available tiles

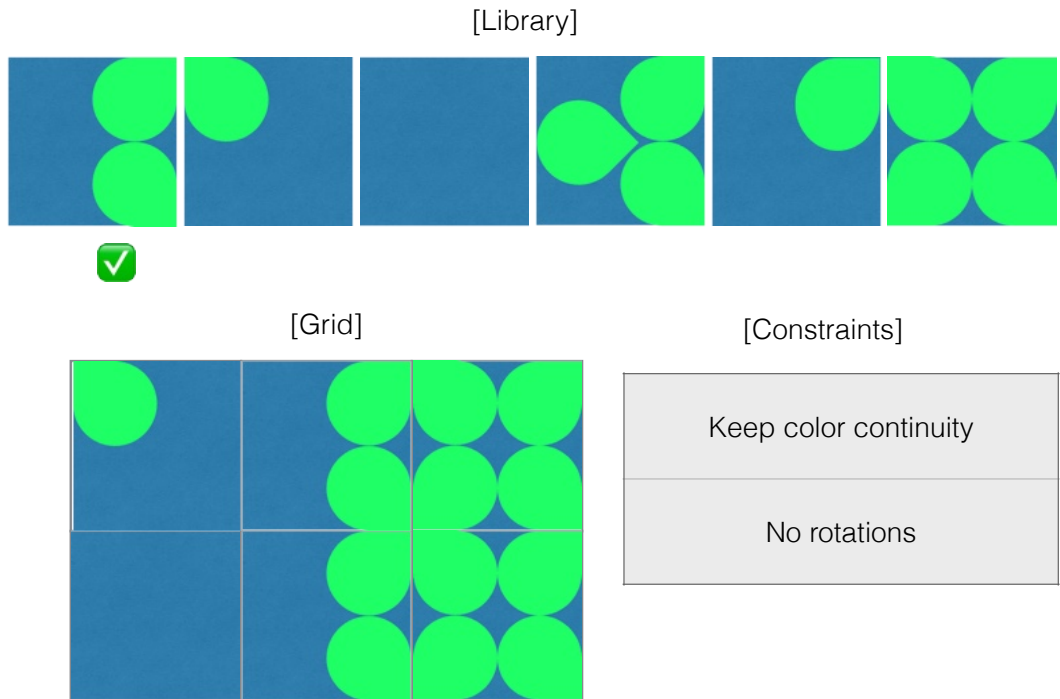
6. Repeat from step 4 until every cell is filled



Wave Function Collapse

- Steps:

1. Define grid and initial cell
2. Define constraints
3. Collapse the initial cell
4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)
5. Collapse cell by assigning one of the available tiles
6. Repeat from step 4 until every cell is filled



Wave Function Collapse

- Steps:

1. Define grid and initial cell

2. Define constraints

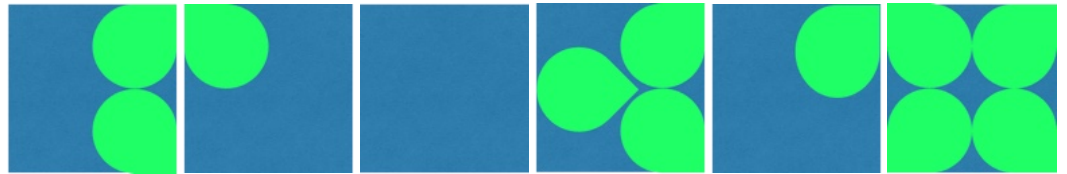
3. Collapse the initial cell

4. Find the cell with lowest entropy level (i.e. how many tiles can be added in each cell)

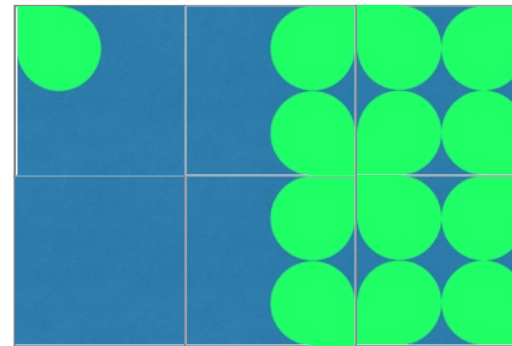
5. Collapse cell by assigning one of the available tiles

6. Repeat from step 4 until every cell is filled

[Library]



[Grid]



[Constraints]

Keep color continuity

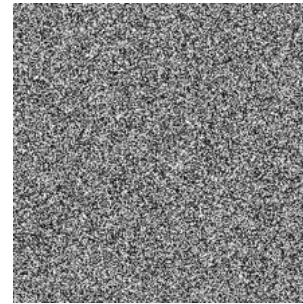
No rotations



Perlin Noise

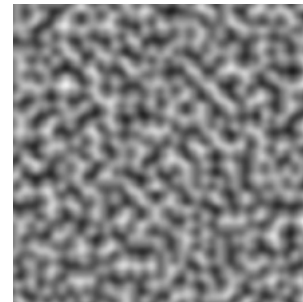
Perlin Noise

- Developed by Ken Perlin while working on the Tron movie to generate textures.
- Unlike pure random noise, Perline Noise is smooth.
- Creates a noise function that can be refined by adding additional noise layers (octaves)

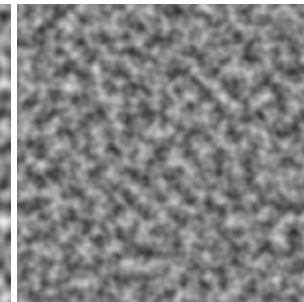


Random (White) Noise

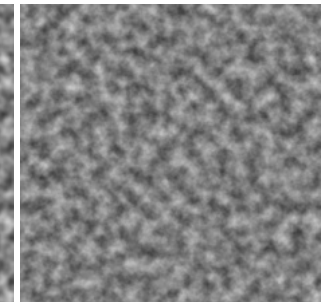
$$\text{Noise}(x, y, \dots) \rightarrow [0,1]$$



1 octave

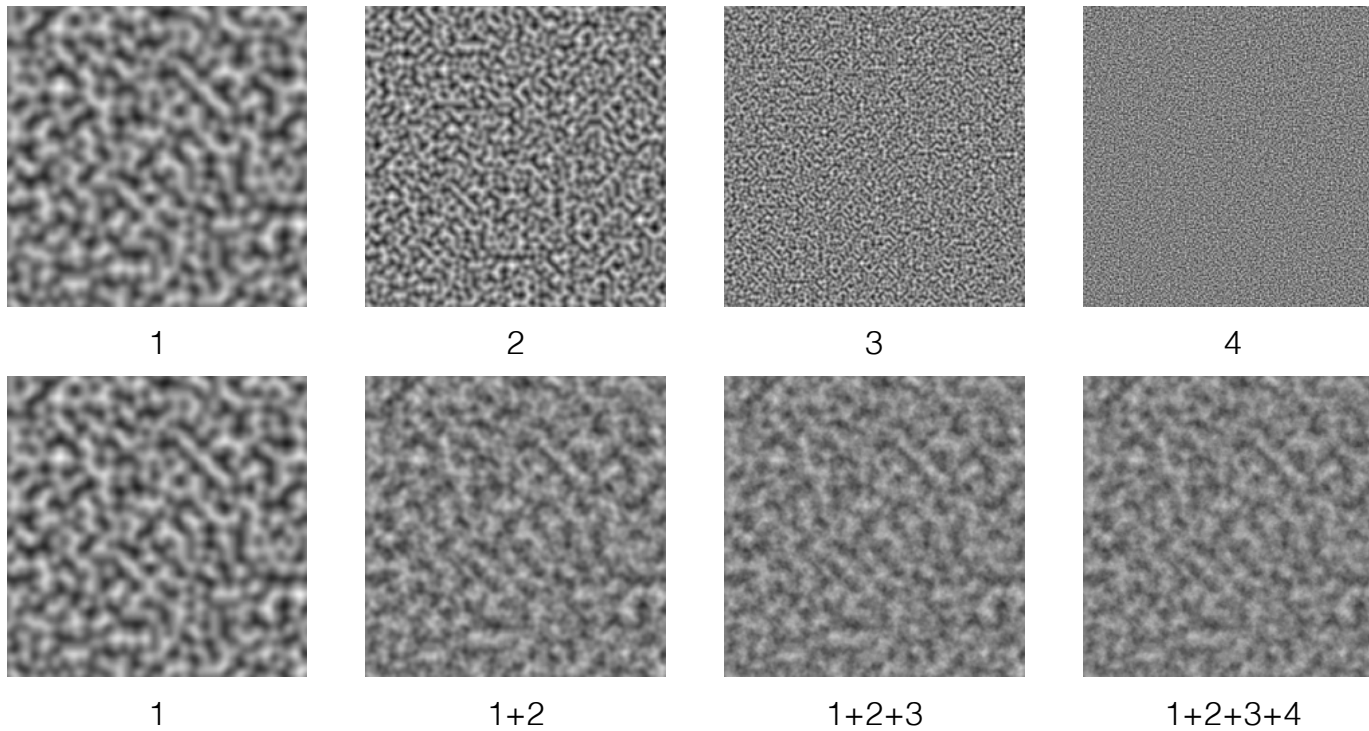


2 octaves



3 octaves

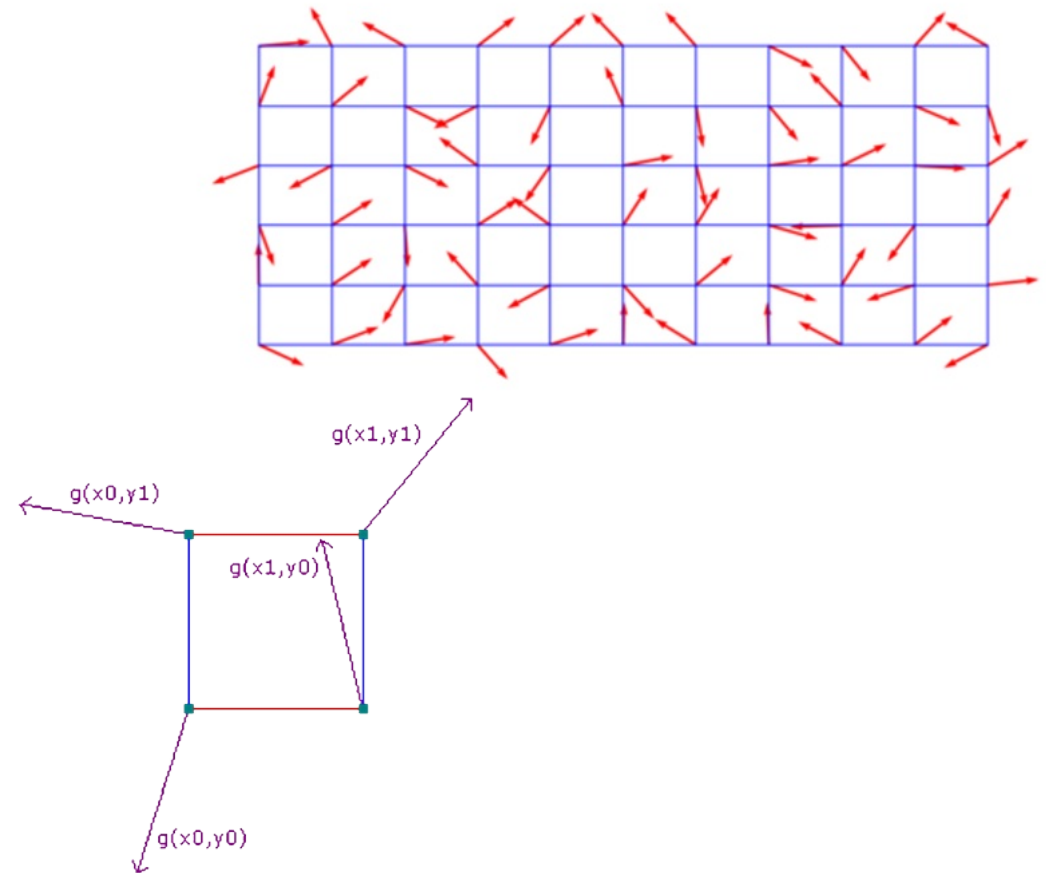
Perlin Noise Octaves



Persistence=0.5 $\Rightarrow$ each octave layer is halved, **Lacunarity**=2.0 $\Rightarrow$ each octave doubles the frequency

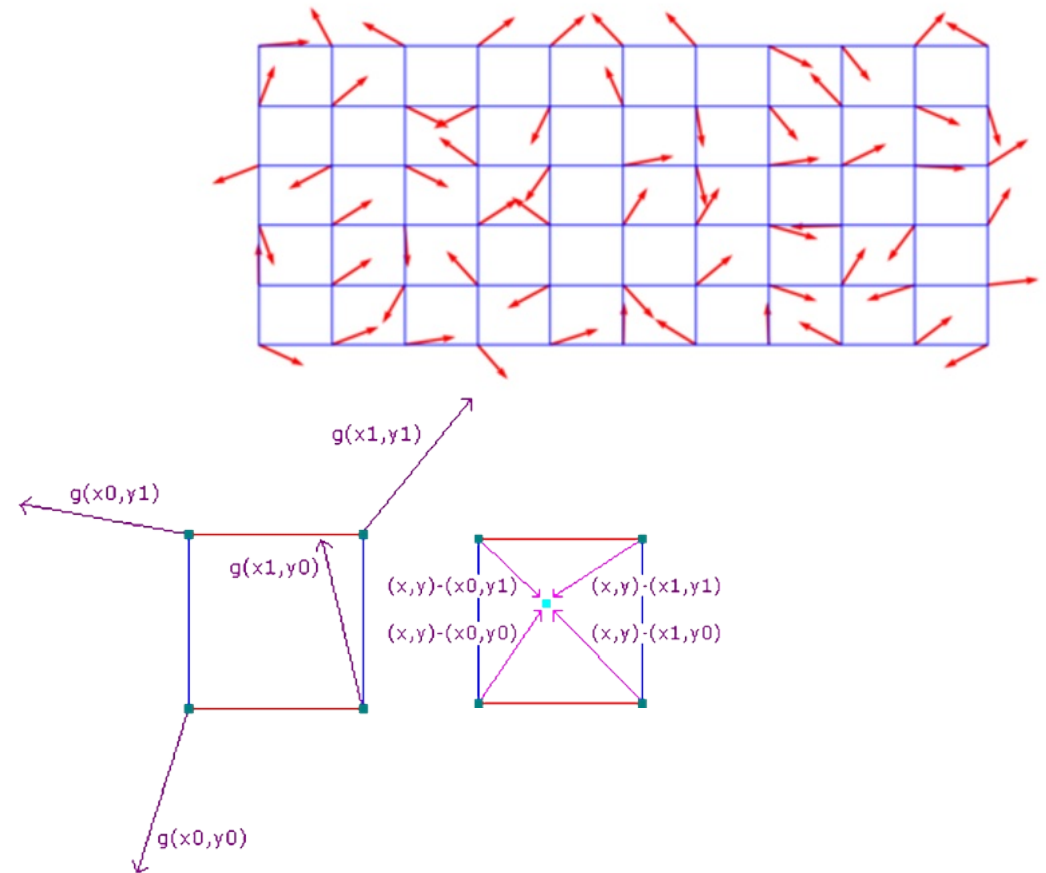
Perlin Noise Algorithm

- Step 1: Define a grid with random unit gradient vectors. These gradients represent the flow at that point.



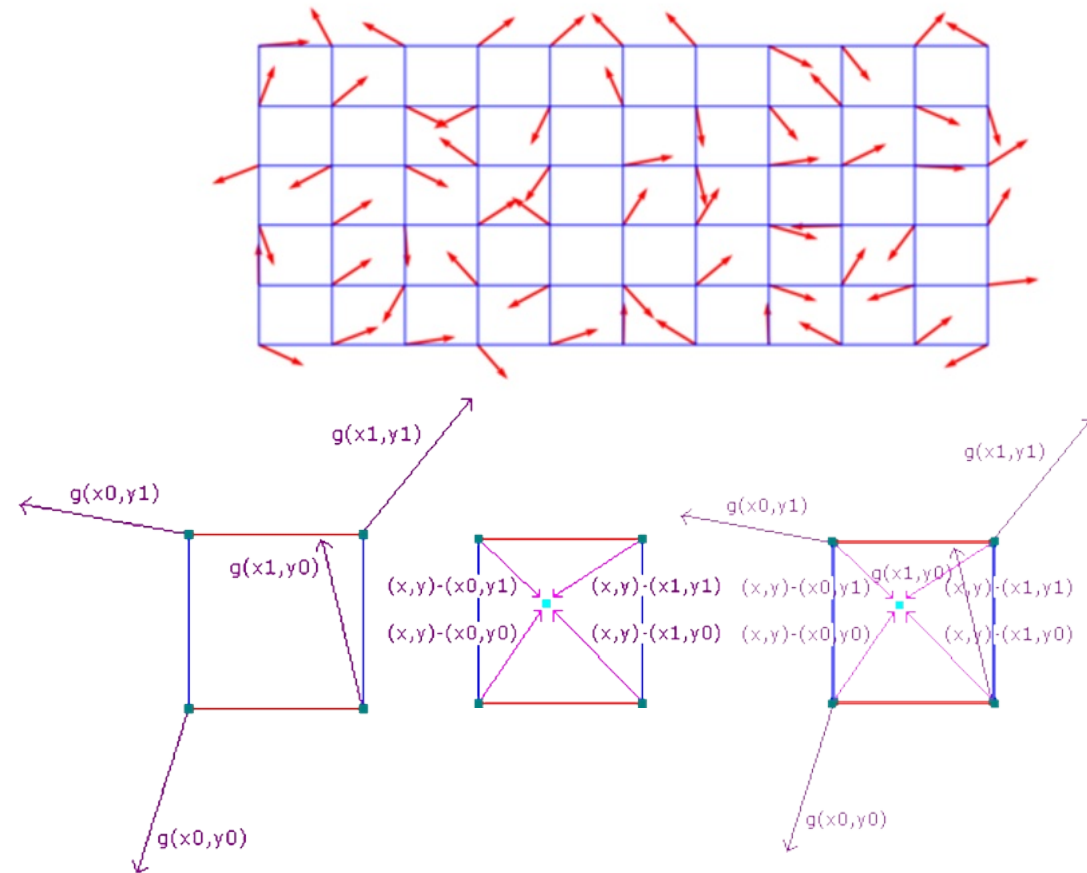
Perlin Noise Algorithm

- Step 1: Define a grid with random unit gradient vectors. These gradients represent the flow at that point.
- Step 2: For a given point $P(x, y)$ where we want to compute the noise, compute the vectors from each corner of the cell that contains point $P(x, y)$



Perlin Noise Algorithm

- Step 1: Define a grid with random unit gradient vectors. These gradients represent the flow at that point.
- Step 2: For a given point $P(x, y)$ where we want to compute the noise, compute the vectors from each corner of the cell that contains point $P(x, y)$
- Step 3: Compute dot product of gradient vectors with distance vectors:
 - The two point in similar directions $\rightarrow$ high value
 - The two are perpendicular $\rightarrow 0$
 - The two point away from each other $\rightarrow$ big negative value



Perlin Noise Algorithm

- Step 4: Interpolate every value in the map (texture to be created) using bi-linear interpolation from the 4 dot product values.

Interpolate between the top two corners using $(x - x_0)$ for t .

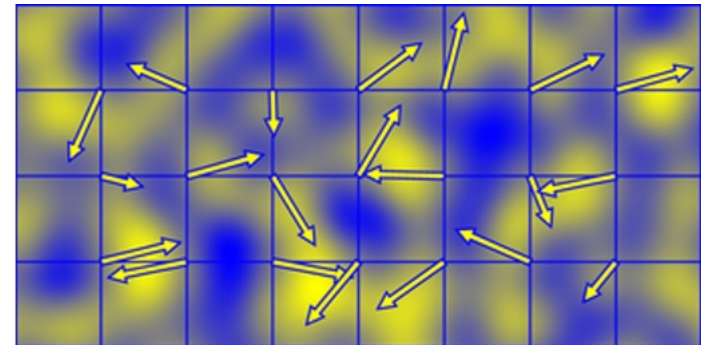
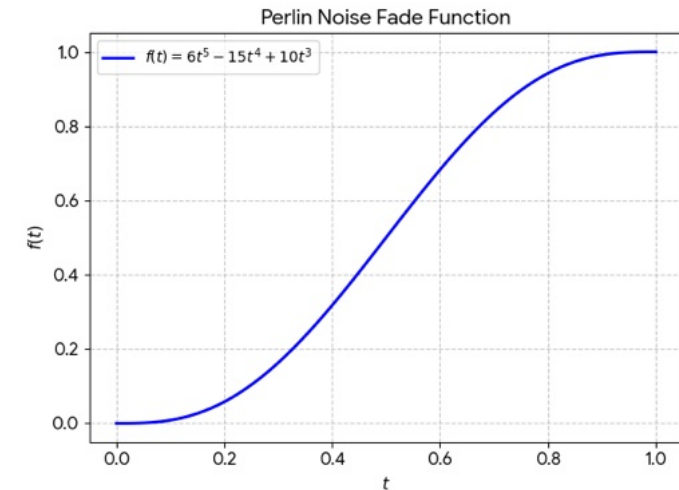
Do the same for the bottom two corners.

Then interpolate between the two newly obtained values using $(y - y_0)$ for t .

Use a “fade” function for interpolation as opposed.

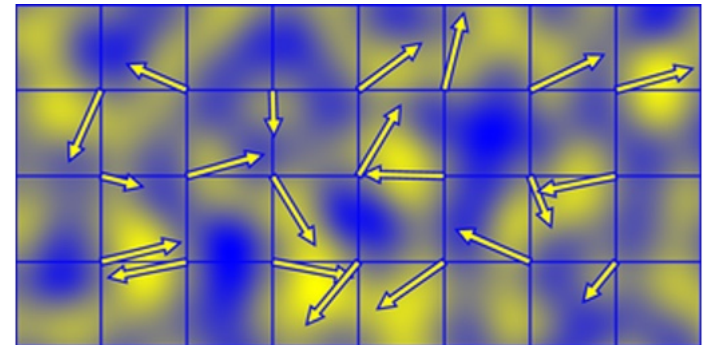
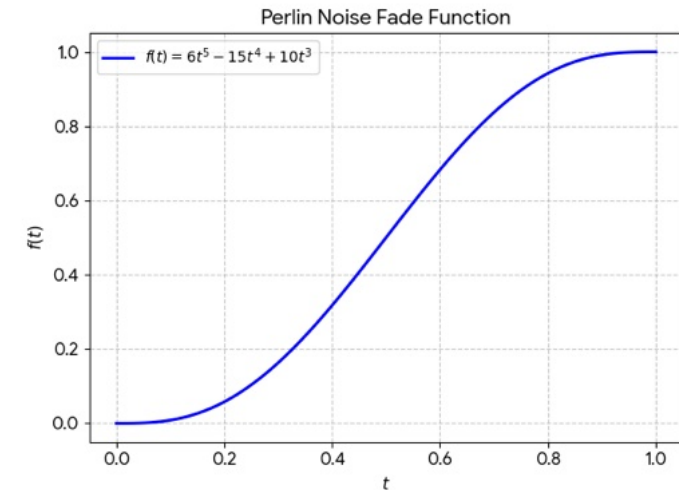
$$f(t) = 6t^5 - 15t^4 + 10t^3$$

What are the values for $f'(0)$ and $f'(1)$? Why?



Perlin Noise Algorithm

- Step 5: Use noise to generate whatever you want:
 - Textures
 - Terrain (height maps)
 - Volumetric clouds (3D noise)
 - Cavern like structures (3D noise)
 - ???



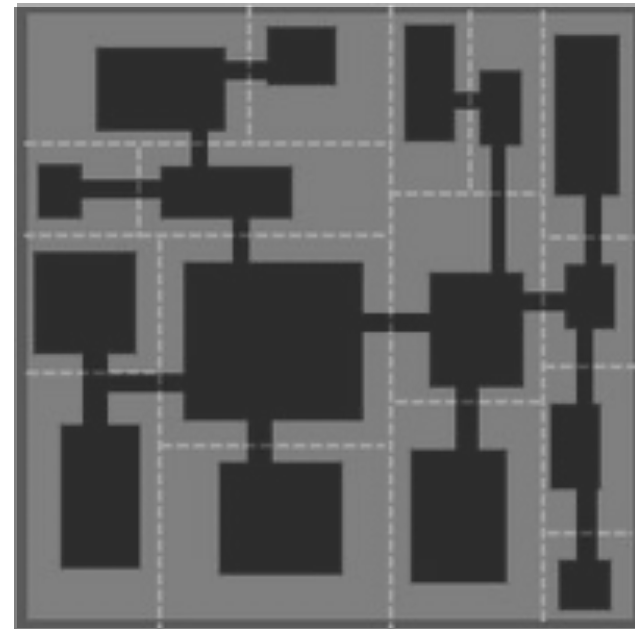
Dungeon Generation

Dungeon Generation Algorithms

- Binary Space Partition (BSP)
- Cellular Automata
- Drunkard's Walk (Random Walk)
- Wave Function Collapse (as seen before)

Binary Space Partition

- **Best for:** Structured, rectangular "man-made" dungeons (think NetHack or Enter the Gungeon).
- BSP works like an architect dividing a large plot of land into smaller and smaller rooms.
- How it works: You start with one large rectangle (the map). You split it into two sub-rectangles (either vertically or horizontally). You repeat this recursively until the rectangles reach a minimum size.
- The Room Phase: Inside each final "leaf" rectangle, you carve out a smaller room of random size.
- The Connection Phase: You traverse back up the tree, connecting sibling nodes with corridors. This guarantees that every room is reachable from every other room.

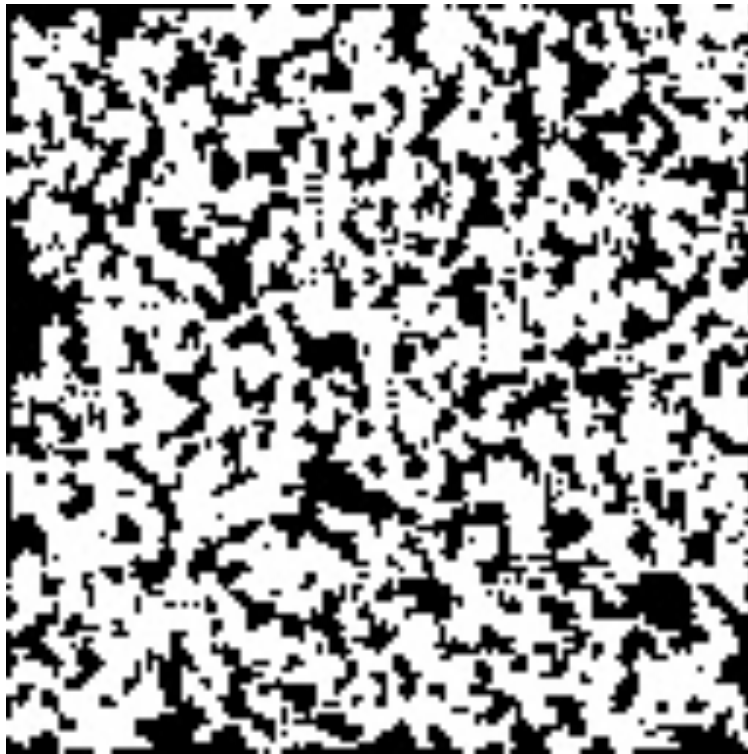


[Source]

Cellular Automata (CA)

- **Best for:** Organic, "natural" caves and rocky caverns.
- Based on the principles of Conway's Game of Life, CA uses a grid of cells that "evolve" over several iterations based on their neighbors.
- **How it works:**
 - Fill the grid randomly with "walls" and "floors" (e.g., 45% walls).
 - For each cell, count how many walls are in its 3x3 neighborhood.
 - The Rule: If a cell has 5 or more wall neighbors, it becomes a wall. Otherwise, it becomes a floor.
 - Repeat this 4–5 times to "smooth" the noise into cavernous shapes.
- The Catch: Unlike BSP, CA can create "islands" or unreachable pockets. You usually need a post-processing step (like a Flood Fill algorithm) to connect or remove isolated areas.

Cellular Automata (CA)

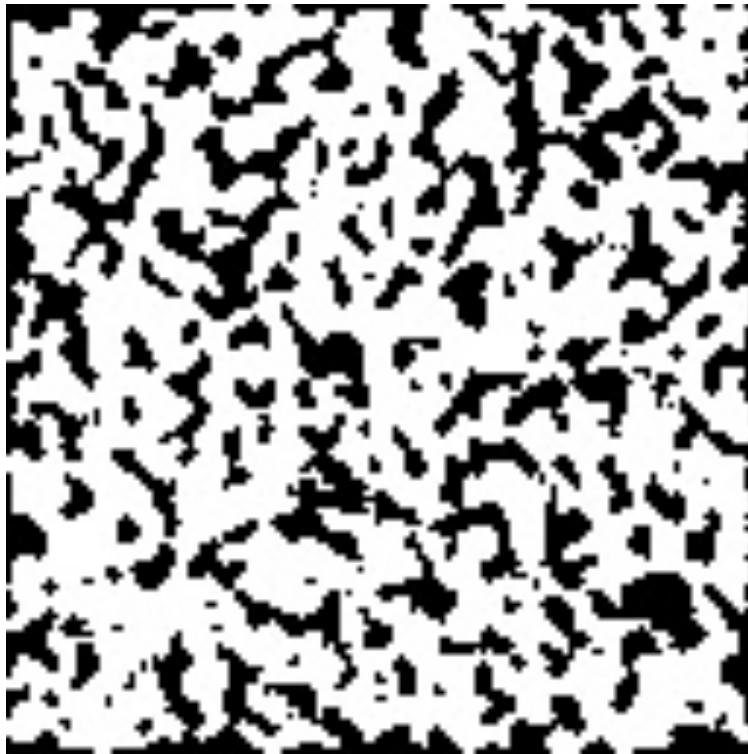


Iteration #1



Walls 45%

Cellular Automata (CA)

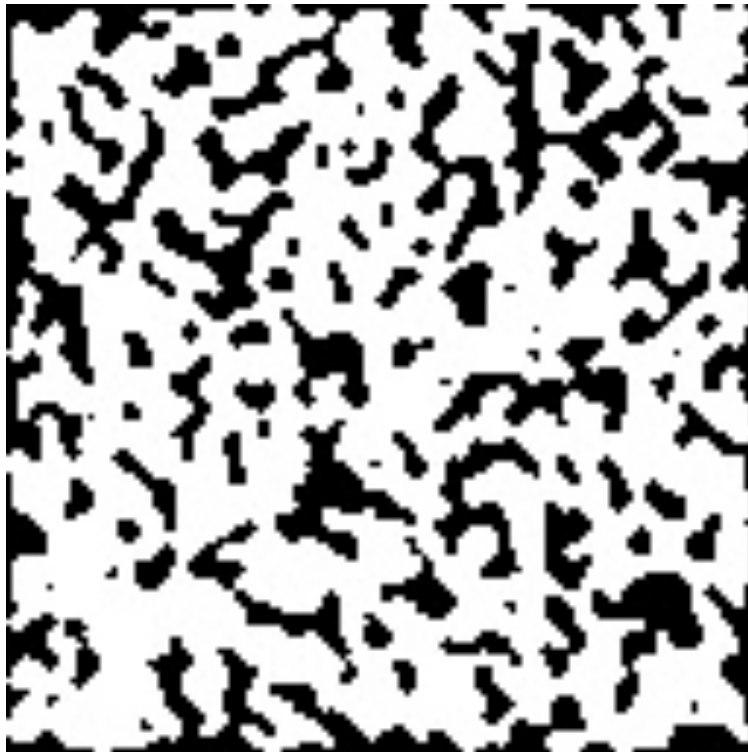


Iteration #2

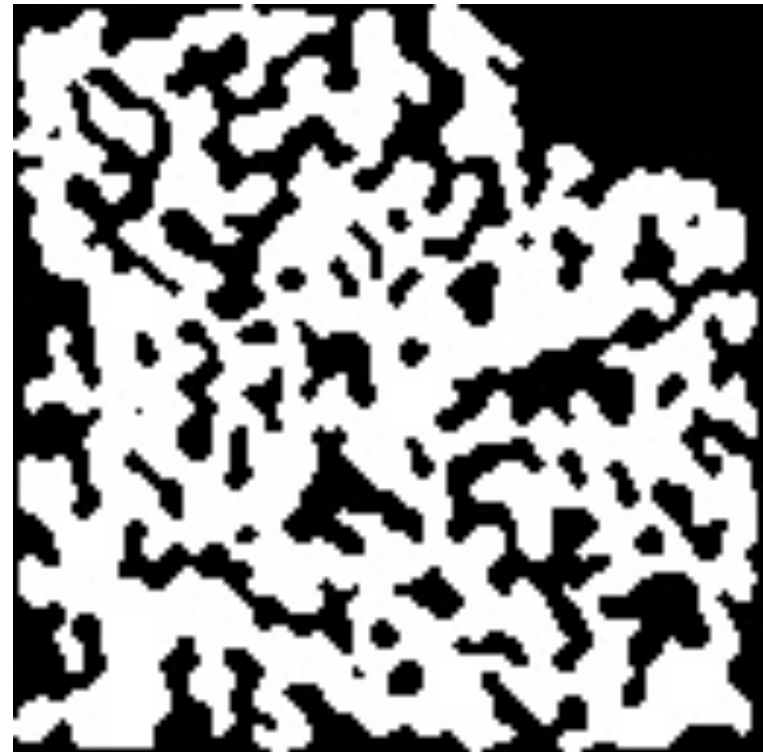


Walls 46%

Cellular Automata (CA)

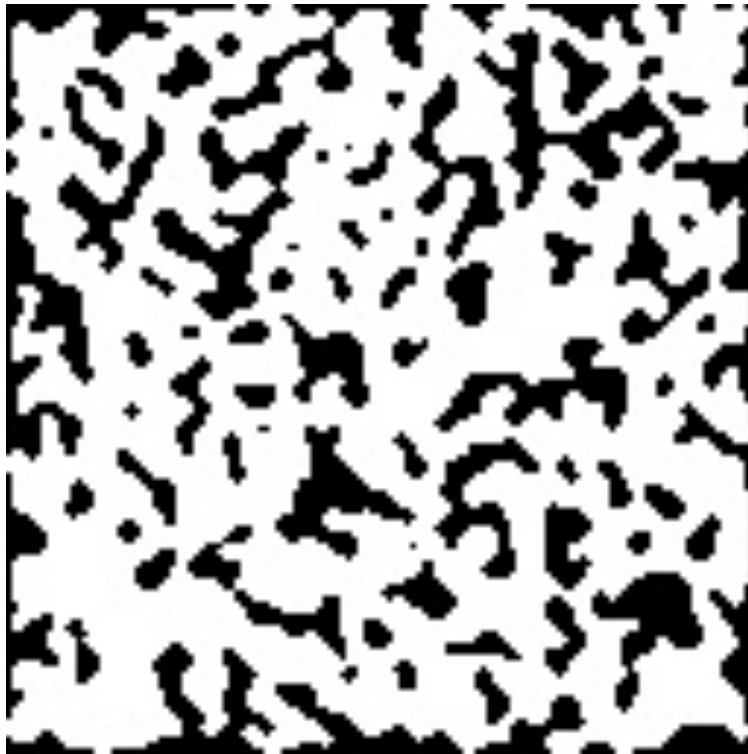


Iteration #3

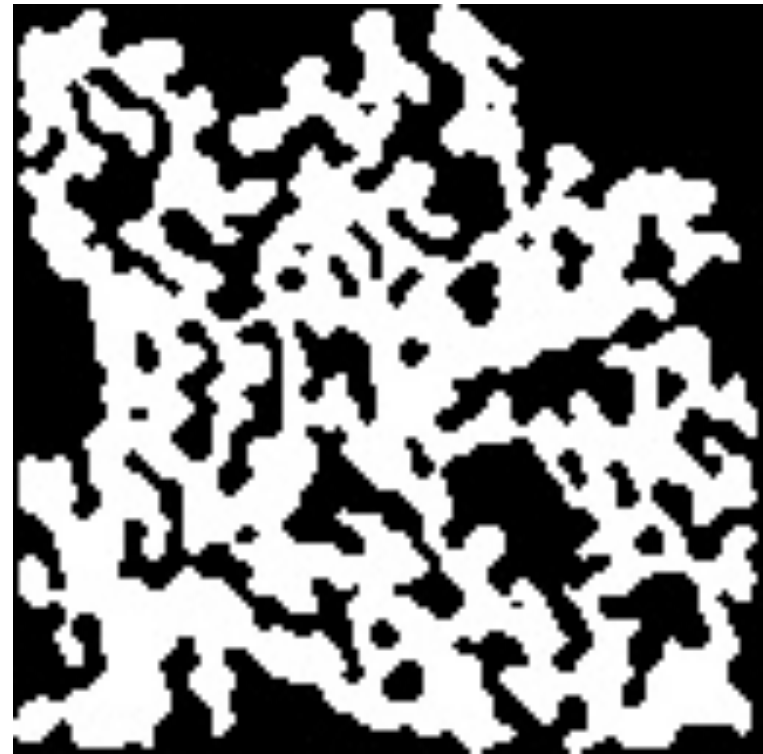


Walls 47%

Cellular Automata (CA)



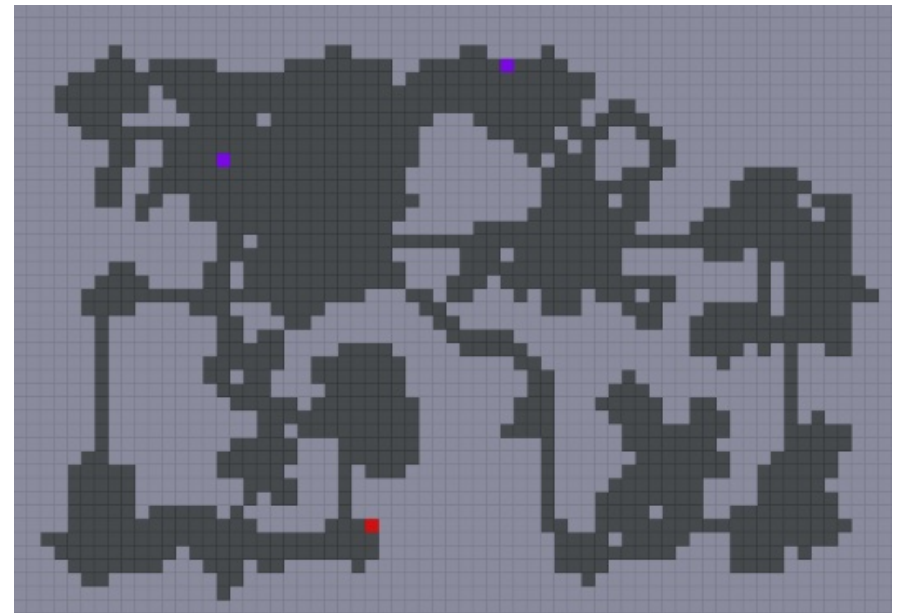
Iteration #4



Walls 48%

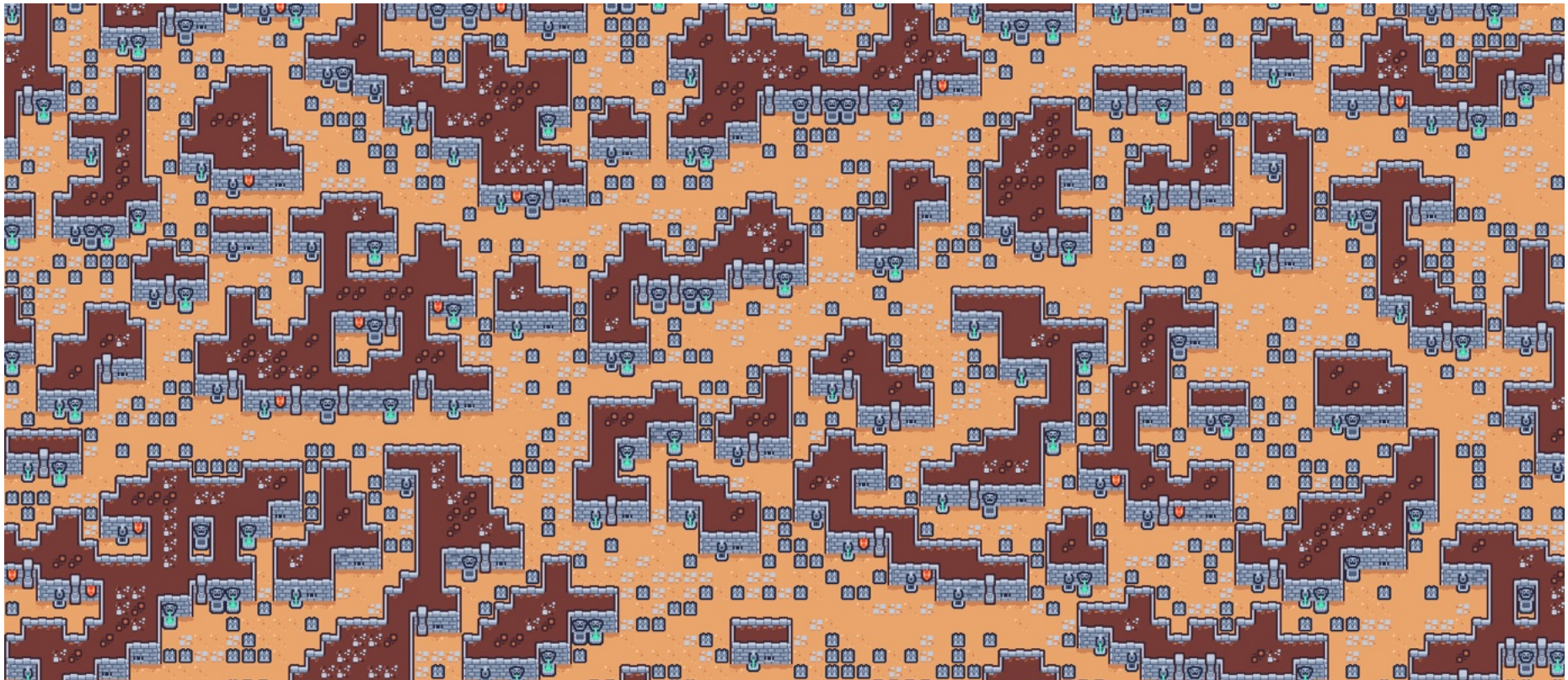
Drunkard's Walk (Random Walk)

- **Best for:** Winding, sprawling tunnels and irregular "ant farm" layouts.
- This is an "agent-based" algorithm where a "digger" starts at a point and wanders aimlessly.
- **How it works:** Pick a starting point. Choose a random direction (North, South, East, West) and move one step, turning that cell into a floor. Repeat this for N steps or until a certain percentage of the map is filled.
- **Pros:** It's incredibly simple to code and guarantees all paths are connected (since the "drunkard" never teleports).
- **Cons:** It can be inefficient, as the agent often walks over the same tiles twice.



[Source]

Wave Function Collapse (WFC)



[Source]